

6. Vue3 核心模块源码解析

1. 课程资料



vue3-sourcecode.zip
2.53MB



2. 课程目标



- 初级：
 - 掌握 Vue3 中响应式及 AST 的使用；
 - 了解编译器基本原理；
 - 了解简单 diff 算法；
- 中级：
 - 掌握编译器原理；
 - 掌握简单 diff 算法、双端 diff 算法；
 - 了解快速 diff 算法；
- 高级：
 - 深入理解 Vue3 核心源码，熟练掌握 Vue3 完整的执行逻辑；
 - 掌握简单 diff 算法、快速 diff 算法、双端 diff 算法；
 - 熟练掌握 Vue2 和 Vue3 diff 算法实现的异同点；

3. 课程大纲

1. Vue3 模块源码解析；
2. Vue 3 Diff算法；
3. 编译器原理

4. Vue3 模块源码解析（整体了解）

3.3.4

官方地址: <https://github.com/vuejs/core/tree/main/packages>

基本核心模块目录结构如下:

```
1  |─compiler-core
2  |   | package.json
3  |   |
4  |   |─src
5  |   |   | ast.ts
6  |   |   | codegen.ts
7  |   |   | compile.ts
8  |   |   | index.ts
9  |   |   | parse.ts
10 |   |   | runtimeHelpers.ts
11 |   |   | transform.ts
12 |   |   | utils.ts
13 |   |   |
14 |   |   |─transforms
15 |   |   |   | transformElement.ts
16 |   |   |   | transformExpression.ts
17 |   |   |   | transformText.ts
18 |   |   |
19 |   |   |─__tests__
20 |   |   |   | codegen.spec.ts
21 |   |   |   | parse.spec.ts
22 |   |   |   | transform.spec.ts
23 |   |   |   |
24 |   |   |   |─__snapshots__
25 |   |   |   |   | codegen.spec.ts.snap
26 |   |   |
27 |   |─reactivity
28 |   |   | package.json
29 |   |   |
30 |   |   |─src
31 |   |   |   | baseHandlers.ts
32 |   |   |   | computed.ts
33 |   |   |   | dep.ts
34 |   |   |   | effect.ts
35 |   |   |   | index.ts
36 |   |   |   | reactive.ts
37 |   |   |   | ref.ts
38 |   |   |   |
39 |   |   |   |─__tests__
40 |   |   |   |   | computed.spec.ts
41 |   |   |   |   | dep.spec.ts
```

```
42 |         effect.spec.ts
43 |         reactive.spec.ts
44 |         readonly.spec.ts
45 |         ref.spec.ts
46 |         shallowReadonly.spec.ts
47 |
48 | └─runtime-core
49 |   |   package.json
50 |   |
51 |   └─src
52 |     |   .pnpm-debug.log
53 |     |   apiInject.ts
54 |     |   apiWatch.ts
55 |     |   component.ts
56 |     |   componentEmits.ts
57 |     |   componentProps.ts
58 |     |   componentPublicInstance.ts
59 |     |   componentRenderUtils.ts
60 |     |   componentSlots.ts
61 |     |   createApp.ts
62 |     |   h.ts
63 |     |   index.ts
64 |     |   renderer.ts
65 |     |   scheduler.ts
66 |     |   vnode.ts
67 |
68 |     └─helpers
69 |       |   renderSlot.ts
70 |
71 |   └─__tests__
72 |     |   apiWatch.spec.ts
73 |     |   componentEmits.spec.ts
74 |     |   rendererComponent.spec.ts
75 |     |   rendererElement.spec.ts
76 |
77 | └─runtime-dom
78 |   |   package.json
79 |   |
80 |   └─src
81 |     |   index.ts
82 |
83 | └─runtime-test
84 |   |   src
85 |     |   index.ts
86 |     |   nodeOps.ts
87 |     |   patchProp.ts
88 |     |   serialize.ts
```

```

89 |
90 | └─shared
91 |   |   package.json
92 |   |
93 |   └─src
94 |       index.ts
95 |       shapeFlags.ts
96 |       toDisplayString.ts

```

4.1 compiler-core

Vue3的编译核心，核心作用就是将字符串转换成 抽象对象语法树AST；

4.1.1 目录结构

```

1 | └─src
2 |   |   ast.ts           // ts类型定义, 比如type, enum, interface等
3 |   |   codegen.ts       // 将生成的ast转换成render字符串
4 |   |   compile.ts      // compile统一执行逻辑, 有一个 baseCompile , 用来编译模板文件的
5 |   |   index.ts        // 入口文件
6 |   |   parse.ts        // 将模板字符串转换成 AST
7 |   |   runtimeHelpers.ts // 生成code时候的定义常量对应关系
8 |   |   transform.ts     // 处理 AST 中的 vue 特有语法
9 |   |   utils.ts
10 |   |
11 |   └─transforms
12 |       transformElement.ts
13 |       transformExpression.ts
14 |       transformText.ts
15 |
16 | └─__tests__           // 测试用例
17 |     |   codegen.spec.ts
18 |     |   parse.spec.ts
19 |     |   transform.spec.ts
20 |     |
21 |     └─__snapshots__
22 |         codegen.spec.ts.snap
23 |

```

4.1.2 compile逻辑

```

1 // src/index.ts

```

```

2  export { baseCompile } from "./compile";
3
4  // src/compiler.ts
5  import { generate } from "./codegen";
6  import { baseParse } from "./parse";
7  import { transform } from "./transform";
8  import { transformExpression } from "./transforms/transformExpression";
9  import { transformElement } from "./transforms/transformElement";
10 import { transformText } from "./transforms/transformText";
11
12 export function baseCompile(template, options) {
13   // 1. 先把 template 也就是字符串 parse 成 ast
14   const ast = baseParse(template);
15   // 2. 给 ast 加点料 (- -#)
16   transform(
17     ast,
18     Object.assign(options, {
19       nodeTransforms: [transformElement, transformText, transformExpression],
20     })
21   );
22
23   // 3. 生成 render 函数代码
24   return generate(ast);
25 }

```

- baseParse

```

1  export function baseParse(content: string) {
2    const context = createParserContext(content);
3    return createRoot(parseChildren(context, []));
4  }
5
6  function createParserContext(content) {
7    console.log("创建 parserContext");
8    return {
9      source: content,
10    };
11  }
12
13  function createRoot(children) {
14    return {
15      type: NodeTypes.ROOT,
16      children,
17      helpers: [],
18    };
19  }

```

```

19 }
20
21 function parseChildren(context, ancestors) {
22     console.log("开始解析 children");
23     const nodes: any = [];
24
25     while (!isEnd(context, ancestors)) {
26         let node;
27         const s = context.source;
28
29         if (startsWith(s, "{")) {
30             // 看看如果是 {{ 开头的话, 那么就是一个插值, 那么去解析他
31             node = parseInterpolation(context);
32         } else if (s[0] === "<") {
33             if (s[1] === "/") {
34                 // 这里属于 edge case 可以不用关心
35                 // 处理结束标签
36                 if (/[a-z]/i.test(s[2])) {
37                     // 匹配 </div>
38                     // 需要改变 context.source 的值 -> 也就是需要移动光标
39                     parseTag(context, TagType.End);
40                     // 结束标签就以为这都已经处理完了, 所以就可以跳出本次循环了
41                     continue;
42                 }
43             } else if (/[a-z]/i.test(s[1])) {
44                 node = parseElement(context, ancestors);
45             }
46         }
47
48         if (!node) {
49             node = parseText(context);
50         }
51
52         nodes.push(node);
53     }
54
55     return nodes;
56 }

```

- transform

```

1 export function transform(root, options = {}) {
2     // 1. 创建 context
3
4     const context = createTransformContext(root, options);

```

```

5
6 // 2. 遍历 node
7 traverseNode(root, context);
8
9 createRootCodegen(root, context);
10
11 root.helpers.push(...context.helpers.keys());
12 }
13
14 function createTransformContext(root, options): any {
15     const context = {
16         root,
17         nodeTransforms: options.nodeTransforms || [],
18         helpers: new Map(),
19         helper(name) {
20             // 这里会收集调用的次数
21             // 收集次数是为了给删除做处理的, (当只有 count 为0 的时候才需要真的删除掉)
22             // helpers 数据会在后续生成代码的时候用到
23             const count = context.helpers.get(name) || 0;
24             context.helpers.set(name, count + 1);
25         },
26     };
27
28     return context;
29 }
30
31 function traverseNode(node: any, context) {
32     const type: NodeTypes = node.type;
33
34     // 遍历调用所有的 nodeTransforms
35     // 把 node 给到 transform
36     // 用户可以对 node 做处理
37     const nodeTransforms = context.nodeTransforms;
38     const exitFns: any = [];
39     for (let i = 0; i < nodeTransforms.length; i++) {
40         const transform = nodeTransforms[i];
41
42         const onExit = transform(node, context);
43         if (onExit) {
44             exitFns.push(onExit);
45         }
46     }
47
48     switch (type) {
49         case NodeTypes.INTERPOLATION:
50             // 插值的点, 在于后续生成 render 代码的时候是获取变量的值
51             context.helper(TO_DISPLAY_STRING);

```

```

52         break;
53
54     case NodeTypes.ROOT:
55     case NodeTypes.ELEMENT:
56
57         traverseChildren(node, context);
58         break;
59
60     default:
61         break;
62 }
63
64
65
66 let i = exitFns.length;
67 // i-- 这个很巧妙
68 // 使用 while 是要比 for 快 (可以使用 https://jsbench.me/ 来测试一下)
69 while (i--) {
70     exitFns[i]();
71 }
72 }
73
74 function createRootCodegen(root: any, context: any) {
75     const { children } = root;
76
77     // 只支持有一个根节点
78     // 并且还是一个 single text node
79     const child = children[0];
80
81     // 如果是 element 类型的话 , 那么我们需要把它的 codegenNode 赋值给 root
82     // root 其实是个空的什么数据都没有的节点
83     // 所以这里需要额外的处理 codegenNode
84     // codegenNode 的目的是专门为了 codegen 准备的 为的就是和 ast 的 node 分离开
85     if (child.type === NodeTypes.ELEMENT && child.codegenNode) {
86         const codegenNode = child.codegenNode;
87         root.codegenNode = codegenNode;
88     } else {
89         root.codegenNode = child;
90     }
91 }
92

```

- generate

```

1 export function generate(ast, options = {}) {

```



```

2    // 先生成 context
3    const context = createCodeGenContext(ast, options);
4    const { push, mode } = context;
5
6    // 1. 先生成 preambleContext
7
8    if (mode === "module") {
9        genModulePreamble(ast, context);
10   } else {
11       genFunctionPreamble(ast, context);
12   }
13
14   const functionName = "render";
15
16   const args = ["_ctx"];
17
18   // _ctx,aaa,bbb,ccc
19   // 需要把 args 处理成 上面的 string
20   const signature = args.join(", ");
21   push(`function ${functionName}(${signature}) {}`);
22   // 这里需要生成具体的代码内容
23   // 开始生成 vnode tree 的表达式
24   push("return ");
25   genNode(ast.codegenNode, context);
26
27   push("}");
28
29   return {
30       code: context.code,
31   };
32 }

```

4.2 reactivity

负责Vue3中响应式实现的部分

4.2.1 目录结构

```

1  |—src
2  |   baseHandlers.ts // 基本处理逻辑
3  |   computed.ts // computed属性处理
4  |   dep.ts // effect对象存储逻辑
5  |   effect.ts // 依赖收集机制
6  |   index.ts // 入口文件
7  |   reactive.ts // 响应式处理逻辑

```

```

8 |         ref.ts // ref执行逻辑
9 |
10 | └─ __tests__ // 测试用例
11 |     computed.spec.ts
12 |     dep.spec.ts
13 |     effect.spec.ts
14 |     reactive.spec.ts
15 |     readonly.spec.ts
16 |     ref.spec.ts
17 |     shallowReadonly.spec.ts

```

4.2.2 reactivity逻辑

- index.ts

```

1  export {
2    reactive,
3    readonly,
4    shallowReadonly,
5    isReadonly,
6    isReactive,
7    isProxy,
8  } from "./reactive";
9
10 export { ref, proxyRefs, unRef, isRef } from "./ref";
11
12 export { effect, stop, ReactiveEffect } from "./effect";
13
14 export { computed } from "./computed";
15

```

- reactive.ts

```

1  import {
2    mutableHandlers,
3    readonlyHandlers,
4    shallowReadonlyHandlers,
5  } from "./baseHandlers";
6
7  export const reactiveMap = new WeakMap();
8  export const readonlyMap = new WeakMap();
9  export const shallowReadonlyMap = new WeakMap();
10

```

```
11 export const enum ReactiveFlags {
12     IS_REACTIVE = "__v_isReactive",
13     IS_READONLY = "__v_isReadonly",
14     RAW = "__v_raw",
15 }
16
17 export function reactive(target) {
18     return createReactiveObject(target, reactiveMap, mutableHandlers);
19 }
20
21 export function readonly(target) {
22     return createReactiveObject(target, readonlyMap, readonlyHandlers);
23 }
24
25 export function shallowReadonly(target) {
26     return createReactiveObject(
27         target,
28         shallowReadonlyMap,
29         shallowReadonlyHandlers
30     );
31 }
32
33 export function isProxy(value) {
34     return isReactive(value) || isReadonly(value);
35 }
36
37 export function isReadonly(value) {
38     return !!value[ReactiveFlags.IS_READONLY];
39 }
40
41 export function isReactive(value) {
42     // 如果 value 是 proxy 的话
43     // 会触发 get 操作, 而在 createGetter 里面会判断
44     // 如果 value 是普通对象的话
45     // 那么会返回 undefined , 那么就需要转换成布尔值
46     return !!value[ReactiveFlags.IS_REACTIVE];
47 }
48
49 export function toRaw(value) {
50     // 如果 value 是 proxy 的话 , 那么直接返回就可以了
51     // 因为会触发 createGetter 内的逻辑
52     // 如果 value 是普通对象的话,
53     // 我们就应该返回普通对象
54     // 只要不是 proxy , 只要是得到了 undefined 的话, 那么就一定是普通对象
55     // TODO 这里和源码里面实现的不一样, 不确定后面会不会有问题
56     if (!value[ReactiveFlags.RAW]) {
57         return value;
```

```

58     }
59
60     return value[ReactiveFlags.RAW];
61 }
62
63 function createReactiveObject(target, proxyMap, baseHandlers) {
64     // 核心就是 proxy
65     // 目的是可以侦听到用户 get 或者 set 的动作
66
67     // 如果命中的话就直接返回就好了
68     // 使用缓存做的优化点
69     const existingProxy = proxyMap.get(target);
70     if (existingProxy) {
71         return existingProxy;
72     }
73
74     const proxy = new Proxy(target, baseHandlers);
75
76     // 把创建好的 proxy 给存起来,
77     proxyMap.set(target, proxy);
78     return proxy;
79 }
80

```

- ref.ts

```

1  import { trackEffects, triggerEffects, isTracking } from "./effect";
2  import { createDep } from "./dep";
3  import { isObject, hasChanged } from "@mini-vue/shared";
4  import { reactive } from "./reactive";
5
6  export class RefImpl {
7      private _rawValue: any;
8      private _value: any;
9      public dep;
10     public __v_isRef = true;
11
12     constructor(value) {
13         this._rawValue = value;
14         // 看看value 是不是一个对象, 如果是一个对象的话
15         // 那么需要用 reactive 包裹一下
16         this._value = convert(value);
17         this.dep = createDep();
18     }
19

```

```
20     get value() {
21         // 收集依赖
22         trackRefValue(this);
23         return this._value;
24     }
25
26     set value(newValue) {
27         // 当新的值不等于老的值的话,
28         // 那么才需要触发依赖
29         if (hasChanged(newValue, this._rawValue)) {
30             // 更新值
31             this._value = convert(newValue);
32             this._rawValue = newValue;
33             // 触发依赖
34             triggerRefValue(this);
35         }
36     }
37 }
38
39 export function ref(value) {
40     return createRef(value);
41 }
42
43 function convert(value) {
44     return isObject(value) ? reactive(value) : value;
45 }
46
47 function createRef(value) {
48     const refImpl = new RefImpl(value);
49
50     return refImpl;
51 }
52
53 export function triggerRefValue(ref) {
54     triggerEffects(ref.dep);
55 }
56
57 export function trackRefValue(ref) {
58     if (isTracking()) {
59         trackEffects(ref.dep);
60     }
61 }
62
63 // 这个函数的目的是
64 // 帮助解构 ref
65 // 比如在 template 中使用 ref 的时候, 直接使用就可以了
66 // 例如: const count = ref(0) -> 在 template 中使用的话 可以直接 count
```

```

67 // 解决方案就是通过 proxy 来对 ref 做处理
68
69 const shallowUnwrapHandlers = {
70   get(target, key, receiver) {
71     // 如果里面是一个 ref 类型的话, 那么就返回 .value
72     // 如果不是的话, 那么直接返回value 就可以了
73     return unRef(Reflect.get(target, key, receiver));
74   },
75   set(target, key, value, receiver) {
76     const oldValue = target[key];
77     if (isRef(oldValue) && !isRef(value)) {
78       return (target[key].value = value);
79     } else {
80       return Reflect.set(target, key, value, receiver);
81     }
82   },
83 };
84
85 // 这里没有处理 objectWithRefs 是 reactive 类型的时候
86 // TODO reactive 里面如果有 ref 类型的 key 的话, 那么也是不需要调用 ref.value 的
87 // (but 这个逻辑在 reactive 里面没有实现)
88 export function proxyRefs(objectWithRefs) {
89   return new Proxy(objectWithRefs, shallowUnwrapHandlers);
90 }
91
92 // 把 ref 里面的值拿到
93 export function unRef(ref) {
94   return isRef(ref) ? ref.value : ref;
95 }
96
97 export function isRef(value) {
98   return !!value.__v_isRef;
99 }
100

```

- effect

```

1 export function effect(fn, options = {}) {
2   const _effect = new ReactiveEffect(fn);
3
4   // 把用户传过来的值合并到 _effect 对象上去
5   // 缺点就是不是显式的, 看代码的时候并不知道有什么值
6   extend(_effect, options);
7   _effect.run();
8

```

```

9    // 把 _effect.run 这个方法返回
10   // 让用户可以自行选择调用的时机 (调用 fn)
11   const runner: any = _effect.run.bind(_effect);
12   runner.effect = _effect;
13   return runner;
14 }
15
16 export function stop(runner) {
17   runner.effect.stop();
18 }

```

- computed

```

1  import { createDep } from "./dep";
2  import { ReactiveEffect } from "./effect";
3  import { trackRefValue, triggerRefValue } from "./ref";
4
5  export class ComputedRefImpl {
6    public dep: any;
7    public effect: ReactiveEffect;
8
9    private _dirty: boolean;
10   private _value
11
12   constructor(getter) {
13     this._dirty = true;
14     this.dep = createDep();
15     this.effect = new ReactiveEffect(getter, () => {
16       // scheduler
17       // 只要触发了这个函数说明响应式对象的值发生了改变
18       // 那么就解锁，后续在调用 get 的时候就会重新执行，所以会得到最新的值
19       if (this._dirty) return;
20
21       this._dirty = true;
22       triggerRefValue(this);
23     });
24   }
25
26   get value() {
27     // 收集依赖
28     trackRefValue(this);
29     // 锁上，只可以调用一次
30     // 当数据改变的时候才会解锁
31     // 这里就是缓存实现的核心
32     // 解锁是在 scheduler 里面做的

```

```

33     if (this._dirty) {
34         this._dirty = false;
35         // 这里执行 run 的话，就是执行用户传入的 fn
36         this._value = this.effect.run();
37     }
38
39     return this._value;
40 }
41 }
42
43 export function computed(getter) {
44     return new ComputedRefImpl(getter);
45 }
46

```

4.3 runtime-core

运行的核心流程，其中包括初始化流程和更新流程

4.3.1 目录结构

```

1  |—src
2  |   |   apiInject.ts           // 提供provider和inject
3  |   |   apiWatch.ts           // 提供watch
4  |   |   component.ts          // 创建组件实例
5  |   |   componentEmits.ts     // 执行组件props 里面的 onXXX 的函数
6  |   |   componentProps.ts     // 获取组件props
7  |   |   componentPublicInstance.ts // 组件通用实例上的代理,如$el,$emit等
8  |   |   componentRenderUtils.ts // 判断组件是否需要重新渲染的工具类
9  |   |   componentSlots.ts     // 组件的slot
10 |   |   createApp.ts           // 根据跟组件创建应用
11 |   |   h.ts                  // 创建节点
12 |   |   index.ts              // 入口文件
13 |   |   renderer.ts           // 渲染机制,包含diff
14 |   |   scheduler.ts          // 触发更新机制
15 |   |   vnode.ts              // vnode节点
16 |   |
17 |   |—helpers
18 |       renderSlot.ts         // 插槽渲染实现
19 |
20 |—__tests__                   // 测试用例
21 |     apiWatch.spec.ts
22 |     componentEmits.spec.ts
23 |     rendererComponent.spec.ts
24 |     rendererElement.spec.ts

```


4.3.2 runtime核心逻辑

- provide/inject

```
1  import { getCurrentInstance } from "../component";
2
3  export function provide(key, value) {
4      const currentInstance = getCurrentInstance();
5
6      if (currentInstance) {
7          let { provides } = currentInstance;
8
9          const parentProvides = currentInstance.parent?.provides;
10
11         // 这里要解决一个问题
12         // 当父级 key 和 爷爷级别的 key 重复的时候, 对于子组件来讲, 需要取最近的父级别组件
13         // 那这里的解决方案就是利用原型链来解决
14         // provides 初始化的时候是在 createComponent 时处理的, 当时是直接把
15         // 所以, 如果说这里发现 provides 和 parentProvides 相等的话, 那么就说明是第一次
16         // 我们就把 parent.provides 作为 currentInstance.provides 的原型重新赋值
17         // 至于为什么不在 createComponent 的时候做这个处理, 可能的好处是在这里初始化的话,
18         // 是有个懒执行的效果 (优化点, 只有需要的时候在初始化)
19         if (parentProvides === provides) {
20             provides = currentInstance.provides = Object.create(parentProvides);
21         }
22         provides[key] = value;
23     }
24 }
25
26 export function inject(key, defaultValue) {
27     const currentInstance = getCurrentInstance();
28
29     if (currentInstance) {
30         const provides = currentInstance.parent?.provides;
31
32         if (key in provides) {
33             return provides[key];
34         } else if (defaultValue) {
35             if (typeof defaultValue === "function") {
36                 return defaultValue();
37             }
38         }
39     }
40 }
```

```

38     return defaultValue;
39   }
40 }
41 }
42

```

- watch

```

1  import { ReactiveEffect } from "@mini-vue/reactivity";
2  import { queuePreFlushCb } from "../scheduler";
3
4  // Simple effect.
5  export function watchEffect(effect) {
6    doWatch(effect);
7  }
8
9  function doWatch(source) {
10     // 把 job 添加到 pre flush 里面
11     // 也就是在视图更新完成之前进行渲染（待确认？）
12     // 当逻辑执行到这里的时候 就已经触发了 watchEffect
13     const job = () => {
14       effect.run();
15     };
16
17     // 这里用 scheduler 的目的就是在更新的时候
18     // 让回调可以在 render 前执行 变成一个异步的行为（这里也可以通过 flush 来改变）
19     const scheduler = () => queuePreFlushCb(job);
20
21     const getter = () => {
22       source();
23     };
24
25     const effect = new ReactiveEffect(getter, scheduler);
26
27     // 这里执行的就是 getter
28     effect.run();
29   }
30

```

- component创建

```

1  export function createComponentInstance(vnode, parent) {
2    const instance = {

```

```

3     type: vnode.type,
4     vnode,
5     next: null, // 需要更新的 vnode, 用于更新 component 类型的组件
6     props: {},
7     parent,
8     provides: parent ? parent.provides : {}, // 获取 parent 的 provides 作为当
    前组件的初始化值 这样就可以继承 parent.provides 的属性了
9     proxy: null,
10    isMounted: false,
11    attrs: {}, // 存放 attrs 的数据
12    slots: {}, // 存放插槽的数据
13    ctx: {}, // context 对象
14    setupState: {}, // 存储 setup 的返回值
15    emit: () => {},
16  };
17
18  // 在 prod 环境下的 ctx 只是下面简单的结构
19  // 在 dev 环境下会更复杂
20  instance.ctx = {
21    _: instance,
22  };
23
24  // 赋值 emit
25  // 这里使用 bind 把 instance 进行绑定
26  // 后面用户使用的时候只需要给 event 和参数即可
27  instance.emit = emit.bind(null, instance) as any;
28
29  return instance;
30 }
31

```

- createApp

```

1  import { createVNode } from "../vnode";
2
3  export function createAppAPI(render) {
4    return function createApp(rootComponent) {
5      const app = {
6        _component: rootComponent,
7        mount(rootContainer) {
8          console.log("基于根组件创建 vnode");
9          const vnode = createVNode(rootComponent);
10         console.log("调用 render, 基于 vnode 进行开箱");
11         render(vnode, rootContainer);
12       },

```

```
13     };
14
15     return app;
16   };
17 }
18
```

- 创建Vnode节点

```
1  import { createVNode } from "./vnode";
2  export const h = (type: any , props: any = null, children: string |
  Array<any> = []) => {
3    return createVNode(type, props, children);
4  };
```

- 入口文件

```
1  export * from "./h";
2  export * from "./createApp";
3  export { getCurrentInstance, registerRuntimeCompiler } from "./component";
4  export { inject, provide } from "./apiInject";
5  export { renderSlot } from "./helpers/renderSlot";
6  export { createTextVNode, createElementVNode } from "./vnode";
7  export { createRenderer } from "./renderer";
8  export { toDisplayString } from "@mini-vue/shared";
9  export {
10    // core
11    reactive,
12    ref,
13    readonly,
14    // utilities
15    unRef,
16    proxyRefs,
17    isReadonly,
18    isReactive,
19    isProxy,
20    isRef,
21    // advanced
22    shallowReadonly,
23    // effect
24    effect,
25    stop,
26    computed,
```

```
27 } from "@mini-vue/reactivity";
```

- render

```
1 // 具体update的Diff见下节课内容;
2 function updateElement(n1, n2, container, anchor, parentComponent) {
3     const oldProps = (n1 && n1.props) || {};
4     const newProps = n2.props || {};
5     // 应该更新 element
6     console.log("应该更新 element");
7     console.log("旧的 vnode", n1);
8     console.log("新的 vnode", n2);
9
10    // 需要把 el 挂载到新的 vnode
11    const el = (n2.el = n1.el);
12
13    // 对比 props
14    patchProps(el, oldProps, newProps);
15
16    // 对比 children
17    patchChildren(n1, n2, el, anchor, parentComponent);
18 }
```

- scheduler

```
1 // 具体的调度机制见下节课内容
2 const queue: any[] = [];
3 const activePreFlushCbs: any = [];
4
5 const p = Promise.resolve();
6 let isFlushPending = false;
7
8 export function nextTick(fn?) {
9     return fn ? p.then(fn) : p;
10 }
11
12 export function queueJob(job) {
13     if (!queue.includes(job)) {
14         queue.push(job);
15         // 执行所有的 job
16         queueFlush();
17     }
18 }
```

```
19
20 function queueFlush() {
21     // 如果同时触发了两个组件的更新的话
22     // 这里就会触发两次 then （微任务逻辑）
23     // 但是着是没有必要的
24     // 我们只需要触发一次即可处理完所有的 job 调用
25     // 所以需要判断一下 如果已经触发过 nextTick 了
26     // 那么后面就不需要再次触发一次 nextTick 逻辑了
27     if (isFlushPending) return;
28     isFlushPending = true;
29     nextTick(flushJobs);
30 }
31
32 export function queuePreFlushCb(cb) {
33     queueCb(cb, activePreFlushCbs);
34 }
35
36 function queueCb(cb, activeQueue) {
37     // 直接添加到对应的列表内就ok
38     // todo 这里没有考虑 activeQueue 是否已经存在 cb 的情况
39     // 然后在执行 flushJobs 的时候就可以调用 activeQueue 了
40     activeQueue.push(cb);
41
42     // 然后执行队列里面所有的 job
43     queueFlush()
44 }
45
46 function flushJobs() {
47     isFlushPending = false;
48
49     // 先执行 pre 类型的 job
50     // 所以这里执行的job 是在渲染前的
51     // 也就意味着执行这里的 job 的时候 页面还没有渲染
52     flushPreFlushCbs();
53
54     // 这里是执行 queueJob 的
55     // 比如 render 渲染就是属于这个类型的 job
56     let job;
57     while ((job = queue.shift())) {
58         if (job) {
59             job();
60         }
61     }
62 }
63
64 function flushPreFlushCbs() {
65     // 执行所有的 pre 类型的 job
```

```

66     for (let i = 0; i < activePreFlushCbs.length; i++) {
67         activePreFlushCbs[i]();
68     }
69 }
70

```

- vnode类型定义及格式规范

```

1  import { ShapeFlags } from "@mini-vue/shared";
2
3  export { createVNode as createElementVNode }
4
5  export const createVNode = function (
6      type: any,
7      props?: any,
8      children?: string | Array<any>
9  ) {
10     // 注意 type 有可能是 string 也有可能是对象
11     // 如果是对象的话, 那么就是用户设置的 options
12     // type 为 string 的时候
13     // createVNode("div")
14     // type 为组件对象的时候
15     // createVNode(App)
16     const vnode = {
17         el: null,
18         component: null,
19         key: props?.key,
20         type,
21         props: props || {},
22         children,
23         shapeFlag: getShapeFlag(type),
24     };
25
26     // 基于 children 再次设置 shapeFlag
27     if (Array.isArray(children)) {
28         vnode.shapeFlag |= ShapeFlags.ARRAY_CHILDREN;
29     } else if (typeof children === "string") {
30         vnode.shapeFlag |= ShapeFlags.TEXT_CHILDREN;
31     }
32
33     normalizeChildren(vnode, children);
34
35     return vnode;
36 };
37

```

```

38 export function normalizeChildren(vnode, children) {
39   if (typeof children === "object") {
40     // 暂时主要是为了标识出 slots_children 这个类型来
41     // 暂时我们只有 element 类型和 component 类型的组件
42     // 所以我们这里除了 element , 那么只要是 component 的话, 那么children 肯定就是
    slots 了
43     if (vnode.shapeFlag & ShapeFlags.ELEMENT) {
44       // 如果是 element 类型的话, 那么 children 肯定不是 slots
45     } else {
46       // 这里就必然是 component 了,
47       vnode.shapeFlag |= ShapeFlags.SLOTS_CHILDREN;
48     }
49   }
50 }
51 // 用 symbol 作为唯一标识
52 export const Text = Symbol("Text");
53 export const Fragment = Symbol("Fragment");
54
55 /**
56  * @private
57  */
58 export function createTextVNode(text: string = " ") {
59   return createVNode(Text, {}, text);
60 }
61
62 // 标准化 vnode 的格式
63 // 其目的是为了 child 支持多种格式
64 export function normalizeVNode(child) {
65   // 暂时只支持处理 child 为 string 和 number 的情况
66   if (typeof child === "string" || typeof child === "number") {
67     return createVNode(Text, null, String(child));
68   } else {
69     return child;
70   }
71 }
72
73 // 基于 type 来判断是什么类型的组件
74 function getShapeFlag(type: any) {
75   return typeof type === "string"
76     ? ShapeFlags.ELEMENT
77     : ShapeFlags.STATEFUL_COMPONENT;
78 }
79

```

4.4 runtime-dom

Vue3靠虚拟dom，实现跨平台的能力，runtime-dom提供一个渲染器，这个渲染器可以渲染虚拟dom节点到指定的容器中；

4.4.1 主要功能

```
1  // 源码里面这些接口是由 runtime-dom 来实现
2  // 这里先简单实现
3
4  import { isOn } from "@mini-vue/shared";
5  import { createRenderer } from "@mini-vue/runtime-core";
6
7  // 后面也修改成和源码一样的实现
8  function createElement(type) {
9    console.log("CreateElement", type);
10    const element = document.createElement(type);
11    return element;
12  }
13
14  function createText(text) {
15    return document.createTextNode(text);
16  }
17
18  function setText(node, text) {
19    node.nodeValue = text;
20  }
21
22  function setElementText(el, text) {
23    console.log("SetElementText", el, text);
24    el.textContent = text;
25  }
26
27  function patchProp(el, key, preValue, nextValue) {
28    // preValue 之前的值
29    // 为了之后 update 做准备
30    // nextValue 当前的值
31    console.log(`PatchProp 设置属性:${key} 值:${nextValue}`);
32    console.log(`key: ${key} 之前的值是:${preValue}`);
33
34    if (isOn(key)) {
35      // 添加事件处理函数的时候需要注意一下
36      // 1. 添加的和删除的必须是一个函数，不然的话 删除不掉
37      //    那么就需要把之前 add 的函数给存起来，后面删除的时候需要用到
38      // 2. nextValue 有可能是匿名函数，当对比发现不一样的时候也可以通过缓存的机制来避免
        注册多次
39      // 存储所有的事件函数
40      const invokers = el._vei || (el._vei = {});
```

```

41     const existingInvoker = invokers[key];
42     if (nextValue && existingInvoker) {
43         // patch
44         // 直接修改函数的值即可
45         existingInvoker.value = nextValue;
46     } else {
47         const eventName = key.slice(2).toLowerCase();
48         if (nextValue) {
49             const invoker = (invokers[key] = nextValue);
50             el.addEventListener(eventName, invoker);
51         } else {
52             el.removeEventListener(eventName, existingInvoker);
53             invokers[key] = undefined;
54         }
55     }
56 } else {
57     if (nextValue === null || nextValue === "") {
58         el.removeAttribute(key);
59     } else {
60         el.setAttribute(key, nextValue);
61     }
62 }
63 }
64
65 function insert(child, parent, anchor = null) {
66     console.log("Insert");
67     parent.insertBefore(child, anchor);
68 }
69
70 function remove(child) {
71     const parent = child.parentNode;
72     if (parent) {
73         parent.removeChild(child);
74     }
75 }
76
77 let renderer;
78
79 function ensureRenderer() {
80     // 如果 renderer 有值的话, 那么以后都不会初始化了
81     return (
82         renderer ||
83         (renderer = createRenderer({
84             createElement,
85             createText,
86             setText,
87             setElementText,

```

```

88     patchProp,
89     insert,
90     remove,
91   )))
92 );
93 }
94
95 export const createApp = (...args) => {
96   return ensureRenderer().createApp(...args);
97 };
98
99 export * from "@vue/runtime-core"
100

```

4.5 runtime-test

可以理解成runtime-dom的延伸，因为runtime-test对外提供的确实是dom环境的测试，方便用于runtime-core的测试；

4.5.1 目录结构

```

1  —src
2  index.ts
3  nodeOps.ts
4  patchProp.ts
5  serialize.ts
6

```

4.5.2 runtime-test核心逻辑

- index.ts

```

1  // 实现 render 的渲染接口
2  // 实现序列化
3  import { createRenderer } from "@mini-vue/runtime-core";
4  import { extend } from "@vue/shared";
5  import { nodeOps } from "./nodeOps";
6  import { patchProp } from "./patchProp";
7
8  export const { render } = createRenderer(extend({ patchProp }, nodeOps));
9
10 export * from "./nodeOps";
11 export * from "./serialize"

```

```
12 export * from '@mini-vue/runtime-core'
```

- nodeOps, 节点定义及操作再runtime-core中的映射

```
1 export const enum NodeTypes {
2   ELEMENT = "element",
3   TEXT = "TEXT",
4 }
5
6 let nodeId = 0;
7 // 这个函数会在 runtime-core 初始化 element 的时候调用
8 function createElement(tag: string) {
9   // 如果是基于 dom 的话 那么这里会返回 dom 元素
10  // 这里是为了测试 所以只需要反正一个对象就可以了
11  // 后面的话 通过这个对象来做测试
12  const node = {
13    tag,
14    id: nodeId++,
15    type: NodeTypes.ELEMENT,
16    props: {},
17    children: [],
18    parentNode: null,
19  };
20
21  return node;
22 }
23
24 function insert(child, parent) {
25   parent.children.push(child);
26   child.parentNode = parent;
27 }
28
29 function parentNode(node) {
30   return node.parentNode;
31 }
32
33 function setElementText(el, text) {
34   el.children = [
35     {
36       id: nodeId++,
37       type: NodeTypes.TEXT,
38       text,
39       parentNode: el,
40     },
41   ];
42 }
```

```

42 }
43
44 export const nodeOps = { createElement, insert, parentNode, setElementText };
45

```

- serialize, 序列化: 把Vnode处理成 string

```

1  // 把 node 给序列化
2  // 测试的时候好对比
3
4  import { NodeTypes } from './nodeOps';
5
6  // 序列化: 把一个对象给处理成 string (进行流化)
7  export function serialize(node) {
8    if (node.type === NodeTypes.ELEMENT) {
9      return serializeElement(node);
10   } else {
11     return serializeText(node);
12   }
13 }
14
15 function serializeText(node) {
16   return node.text;
17 }
18
19 export function serializeInner(node) {
20   // 把所有节点变成一个string
21   return node.children.map((c) => serialize(c)).join('');
22 }
23
24 function serializeElement(node) {
25   // 把 props 处理成字符串
26   // 规则:
27   // 如果 value 是 null 的话 那么直接返回 ``
28   // 如果 value 是 `` 的话, 那么返回 key
29   // 不然的话返回 key = value (这里的值需要字符串化)
30   const props = Object.keys(node.props)
31     .map((key) => {
32       const value = node.props[key];
33       return value == null
34         ? ``
35         : value === ``
36         ? key
37         : `${key}=${JSON.stringify(value)}`;
38     })

```

```

39     .filter(Boolean)
40     .join(" ");
41
42     console.log("node-----", node.children);
43     return `<${node.tag}${props ? ` ${props}` : ``}>${serializeInner(node)}</${
44         node.tag
45     }>`;
46 }
47

```

4.6 shared

公用逻辑

4.6.1 具体逻辑

```

1  export * from '../src/shapeFlags';
2  export * from '../src/toDisplayString';
3
4  export const isObject = val => {
5      return val !== null && typeof val === 'object';
6  };
7
8  export const isString = val => typeof val === 'string';
9
10 const camelizeRE = /-(\w)/g;
11 /**
12  * @private
13  * 把中划线命名方式转换成驼峰命名方式
14  */
15 export const camelize = (str: string): string => {
16     return str.replace(camelizeRE, (_, c) => (c ? c.toUpperCase() : ''));
17 };
18
19 export const extend = Object.assign;
20
21 // 必须是 on+一个大写字母的格式开头
22 export const isOn = key => /^on[A-Z]/.test(key);
23
24 export function hasChanged(value, oldValue) {
25     return !Object.is(value, oldValue);
26 }
27
28 export function hasOwn(val, key) {
29     return Object.prototype.hasOwnProperty.call(val, key);
30 }

```

```
30 }
31
32 /**
33  * @private
34  * 首字母大写
35  */
36 export const capitalize = (str: string) => str.charAt(0).toUpperCase() +
  str.slice(1);
37
38 /**
39  * @private
40  * 添加 on 前缀, 并且首字母大写
41  */
42 export const toHandlerKey = (str: string) => (str ? `on${capitalize(str)}` :
  ``);
43
44 // 用来匹配 kebab-case 的情况
45 // 比如 onTest-event 可以匹配到 T
46 // 然后取到 T 在前面加一个 - 就可以
47 // \BT 就可以匹配到 T 前面是字母的位置
48 const hyphenateRE = /\B([A-Z])/g;
49 /**
50  * @private
51  */
52 export const hyphenate = (str: string) => str.replace(hyphenateRE,
  '-$1').toLowerCase();
53
54
55 // 组件的类型
56 export const enum ShapeFlags {
57   // 最后要渲染的 element 类型
58   ELEMENT = 1,
59   // 组件类型
60   STATEFUL_COMPONENT = 1 << 2,
61   // vnode 的 children 为 string 类型
62   TEXT_CHILDREN = 1 << 3,
63   // vnode 的 children 为数组类型
64   ARRAY_CHILDREN = 1 << 4,
65   // vnode 的 children 为 slots 类型
66   SLOTS_CHILDREN = 1 << 5
67 }
68
69 export const toDisplayString = (val) => {
70   return String(val);
71 };
72
```

我们需要关注的核心包

compiler-core

- compiler-core
 - parse, 模板字符串转为基础 ast
 - transform 进行转换
 - codegen 生成目标代码

模板什么时候是字符串, 什么时候可能不是字符串

`h('div', null)`

编译器核心代码实现

```
1  // 初步处理, 通过 parser 将模板字符串转为 ast
2  const ast = isString(template) ? baseParse(template, options) : template
3
4  // 通过转换器将 ast 转为目标代码 ast
5  transform(
6    ast,
7    extend({})
8  )
9
10 // 通过目标代码 ast 生成目标代码
11 generate(
12   ast,
13   extend({})
14 )
```

补充: 如果你以后需要自定义编译器实现低代码特性语言的支持 (DSL)

Vue3 编译器的核心代码

reactivity

核心, 针对于不同的响应式对象, vue3 reactivity 源码做了一些什么处理

通过 `createReactiveObject` 创建响应式对象，但是创建响应式有很多规则的

- 基础类型，不需要通过我们这个响应式对象进行代理的

```
1  if (!isObject(target)) {  
2    return target  
3  }
```

Q：那针对于基础类型的，难道不能用 reactive 了吗？

```
1  ref('ffff')  
2  reactive({value: 'ffff'})  
3  
4  const count = ref(0)  
5  
6  console.log(count) // { value: 0 }
```

- weakMap 用来存储响应式对象的引用关系

```
1  const proxy = new Proxy(  
2    target,  
3    targetType === TargetType.COLLECTION ? collectionHandlers : baseHandlers  
4  )  
5  proxyMap.set(target, proxy)
```

runtime-core

runtime 包代表着什么？

js 解释型语言，所以本身其实不存在编译过程的，在处理源码的时候，我们认为分了两个时机：

- 编译时
- 运行时

预先将源码中 `<div>hello</div> -> h('div', 'hello')` 就像是编译，项目跑起来的时候，就直接运行了。

Taro、uni-app。

一套代码，你在写的时候就那一套，然后你在写完以后会打个包，这时候打包的过程我们就当做为编译时。

你针对不同端，将项目跑起来，这就是运行时。

运行时主要就是将编译后的内容进行进一步加工，比如针对于生命周期的处理、watch、directives 等相关处理

```
1 export const onBeforeMount = createHook(LifecycleHooks.BEFORE_MOUNT)
2 export const onMounted = createHook(LifecycleHooks.MOUNTED)
3 export const onBeforeUpdate = createHook(LifecycleHooks.BEFORE_UPDATE)
4 export const onUpdated = createHook(LifecycleHooks.UPDATED)
5 export const onBeforeUnmount = createHook(LifecycleHooks.BEFORE_UNMOUNT)
6 export const onUnmounted = createHook(LifecycleHooks.UNMOUNTED)
7 export const onServerPrefetch = createHook(LifecycleHooks.SERVER_PREFETCH)
```

这里重点认识一下两个编程思路

1. 对外统一口径，设计一致化的 API
2. 策略模式处理多场景

runtime-dom

针对 dom（浏览器环境）的运行时

它是为了补充 runtime-core 针对于浏览器环境运行时的一些特殊处理，比如我可以针对 dom 操作相关（浏览器环境）定义一些内置指令——vModel、vOn、vShow 特殊处理

> 这里重点提示一下，vIf 指令呢，属于编译时处理的

补充说明

Vue3 中的编译时和运行时

- 编译时，就是将 xxx.vue 进行解析，解析生成对应 js 可运行代码
- 运行时，就是将编译时生成的代码进一步处理，关于生命周期的声明、watch

5. Vue 3 Diff算法

简单 diff（只是教学）

patch

在上一节中，我们通过减少 DOM 操作的次数，提升了更新性能。但这种方式仍然存在可优化的空间。举个例子，假设新旧两组子节点 的内容如下：

```
1  const oldChildren = [  
2    { type: 'p' }, { type: 'div' }, { type: 'span' }  
3  ]  
4  const newChildren = [  
5    { type: 'span' }, { type: 'p' }, { type: 'div' }  
6  ]
```

如果使用上一节介绍的算法来完成上述两组子节点的更新，则需要 6 次 DOM 操作。

- 调用 patch 函数在旧子节点 `{ type: 'p' }` 与新子节点 `{ type: 'span' }` 之间打补丁，由于两者是不同的标签，所以 patch 函数会卸载 `{ type: 'p' }` 了，然后再挂载 `{ type: 'span' }` 了，这需要执行 2 次 DOM 操作。
- 与第 1 步类似，卸载旧子节点 `{ type: 'div' }` 了，然后再挂载新子节点 `{ type: 'p' }` 了，这也需要执行 2 次 DOM 操作。
- 与第 1 步类似，卸载旧子节点 `{ type: 'span' }`，然后再挂载新子节点 `{ type: 'div' }` 了，同样需要执行 2 次 DOM 操作。

因此，一共进行 6 次 DOM 操作才能完成上述案例的更新。但是，观察新旧两组子节点，很容易发现，二者只是顺序不同。所以最优的处理方式是，通过 DOM 的移动来完成子节点的更新，这要比不断地执行子节点的卸载和挂载性能更好。但是，想要通过 DOM 的移动来完成更新，必须要保证一个前提：新旧两组子节点中的确存在可复用的节点。这个很好理解，如果新的子节点没有在旧的一组子节点中出现，就无法通过移动节点的方式完成更新。所以现在问题变成了：应该如何确定新的子节点是否出现在旧的一组子节点中呢？拿上面的例子来说，怎么确定新的一组子节点中第 1 个子节点 ^

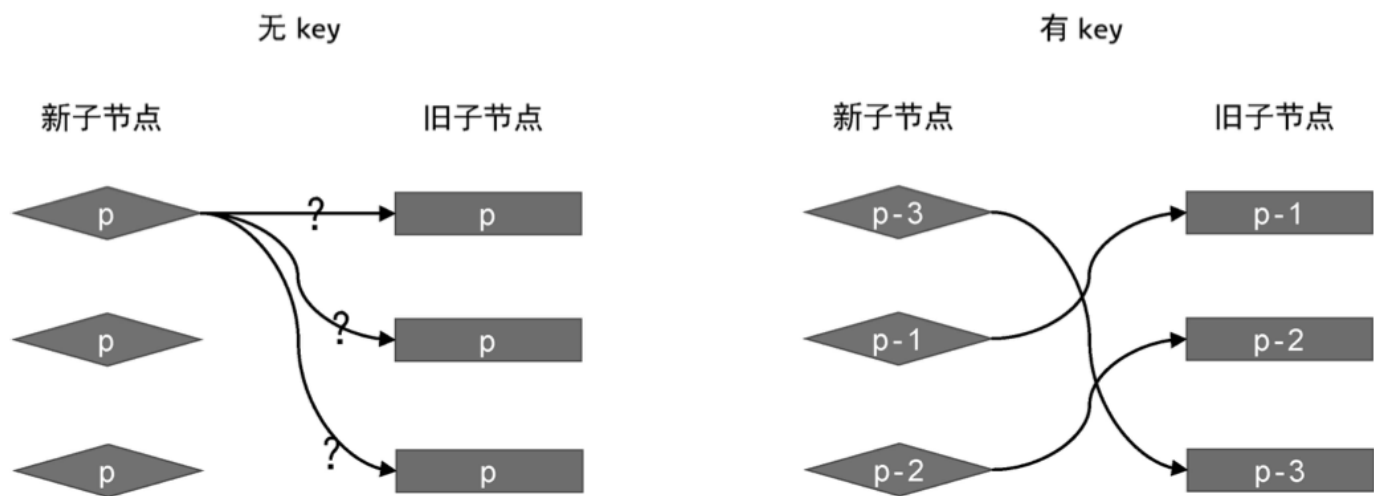
type 'span' 与旧的一组子节点中第 3 个子节点相同呢？一种解决方案是，通过 vnode.type 来判断，只要 vnode.type 的值相同，我们就认为两者是相同的节点。但这种方式并不可靠，思考如下例子：

```
1  const oldChildren = [  
2    { type: 'p', children: '1' },  
3    { type: 'p', children: '2' },  
4    { type: 'p', children: '3' }  
5  ]  
6  const newChildren = [  
7    { type: 'p', children: '3' },  
8    { type: 'p', children: '1' },  
9    { type: 'p', children: '2' }  
10 ]
```

观察上面两组子节点，我们发现，这个案例可以通过移动 DOM 的方式来完成更新。但是所有节点的 vnode.type 属性值都相同，这导致我们无法确定新旧两组子节点中节点的对应关系，也就无法得知应该进行怎样的 DOM 移动才能完成更新。这时，我们就需要引入额外的 key 来作为 vnode 的标识，如下面的代码所示：

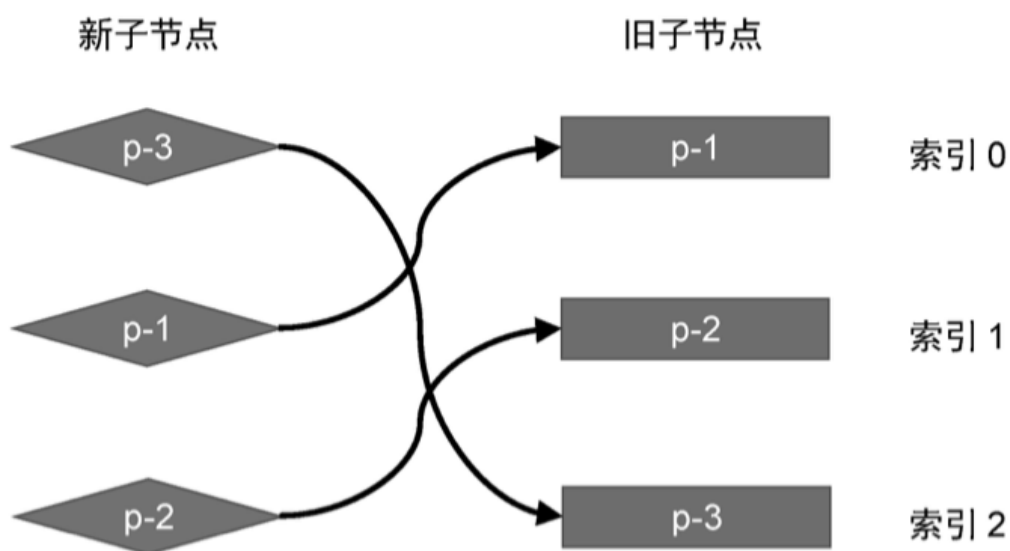
```
1  const oldChildren = [  
2    { type: 'p', children: '1', key: '1' },  
3    { type: 'p', children: '2', key: '2' },  
4    { type: 'p', children: '3', key: '3' }  
5  ]  
6  const newChildren = [  
7    { type: 'p', children: '3', key: '3' },  
8    { type: 'p', children: '1', key: '1' },  
9    { type: 'p', children: '2', key: '2' }  
10 ]  
11
```

key 属性就像虚拟节点的“身份证”号，只要两个虚拟节点的 type 属性值和 key 属性值都相同，那么我们就认为它们是相同的，即可以进行 DOM 的复用。图 9-4 展示了有 key 和无 key 时新旧两组子节点的映射情况。



接下来会进行 patch 操作，patch 理解为打补丁，这个操作是不需要移动元素的，而只需要更新元素即可，不改变顺序。

接下来找寻需要移动的元素，进行元素移动

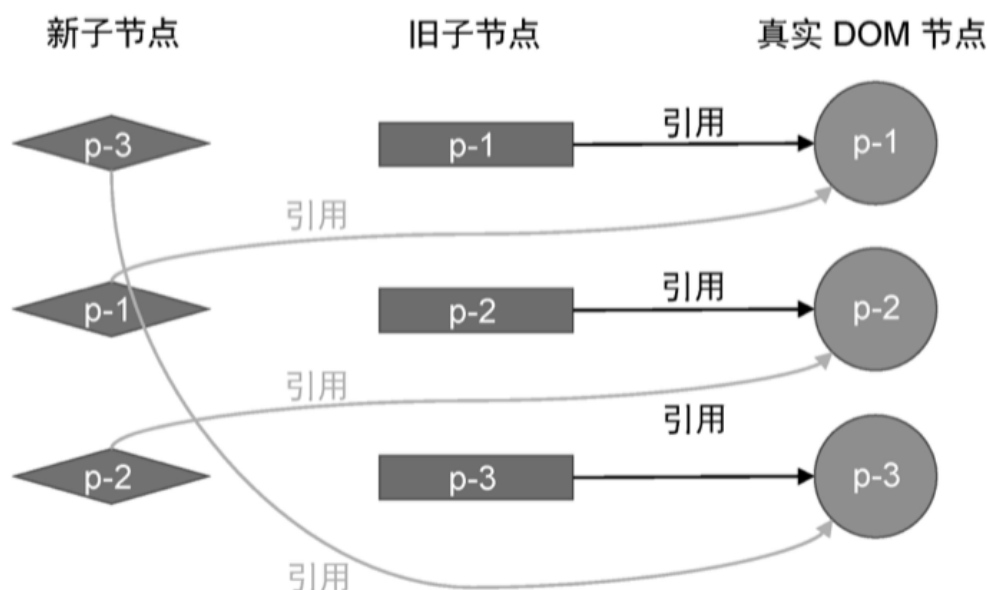


详述整个过程

- 第一步：取新的一组子节点中的第一个节点 p-3，它的 key 为 3。尝试在旧的一组子节点中找到具有相同 key 值的可复用节点，发现能够找到，并且该节点在旧的一组子节点中的索引为 2。

- 第二步：取新的一组子节点中的第二个节点 p-1，它的 key 为 1。尝试在旧的一组子节点中找到具有相同 key 值的可复用节点，发现能够找到，并且该节点在旧的一组子节点中的索引为 0。到了这一步我们发现，索引值递增的顺序被打破了。节点 p-1 在旧 children 中的索引是 0，它小于节点 p-3 在旧 children 中的索引 2。这说明节点 p-1 在旧 children 中排在节点 p-3 前面，但在新的 children 中，它排在节点 p-3 后面。因此，我们能够得出一个结论：节点 p-1 对应的真实 DOM 需要移动。
- 第三步：取新的一组子节点中的第三个节点 p-2，它的 key 为 2。尝试在旧的一组子节点中找到具有相同 key 值的可复用节点，发现能够找到，并且该节点在旧的一组子节点中的索引为 1。到了这一步我们发现，节点 p-2 在旧 children 中的索引 1 要小于节点 p-3 在旧 children 中的索引 2。这说明，节点 p-2 在旧 children 中排在节点 p-3 前面，但在新的 children 中，它排在节点 p-3 后面。因此，节点 p-2 对应的真实 DOM 也需要移动。

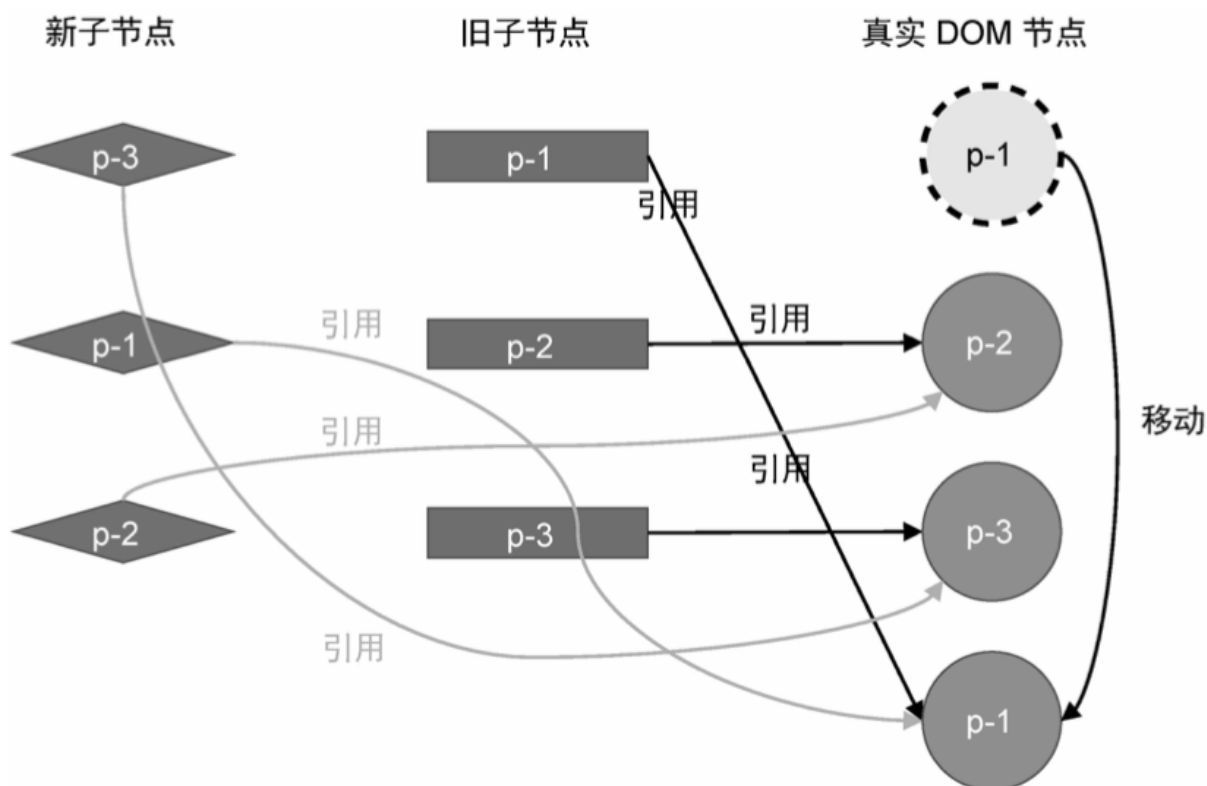
复用



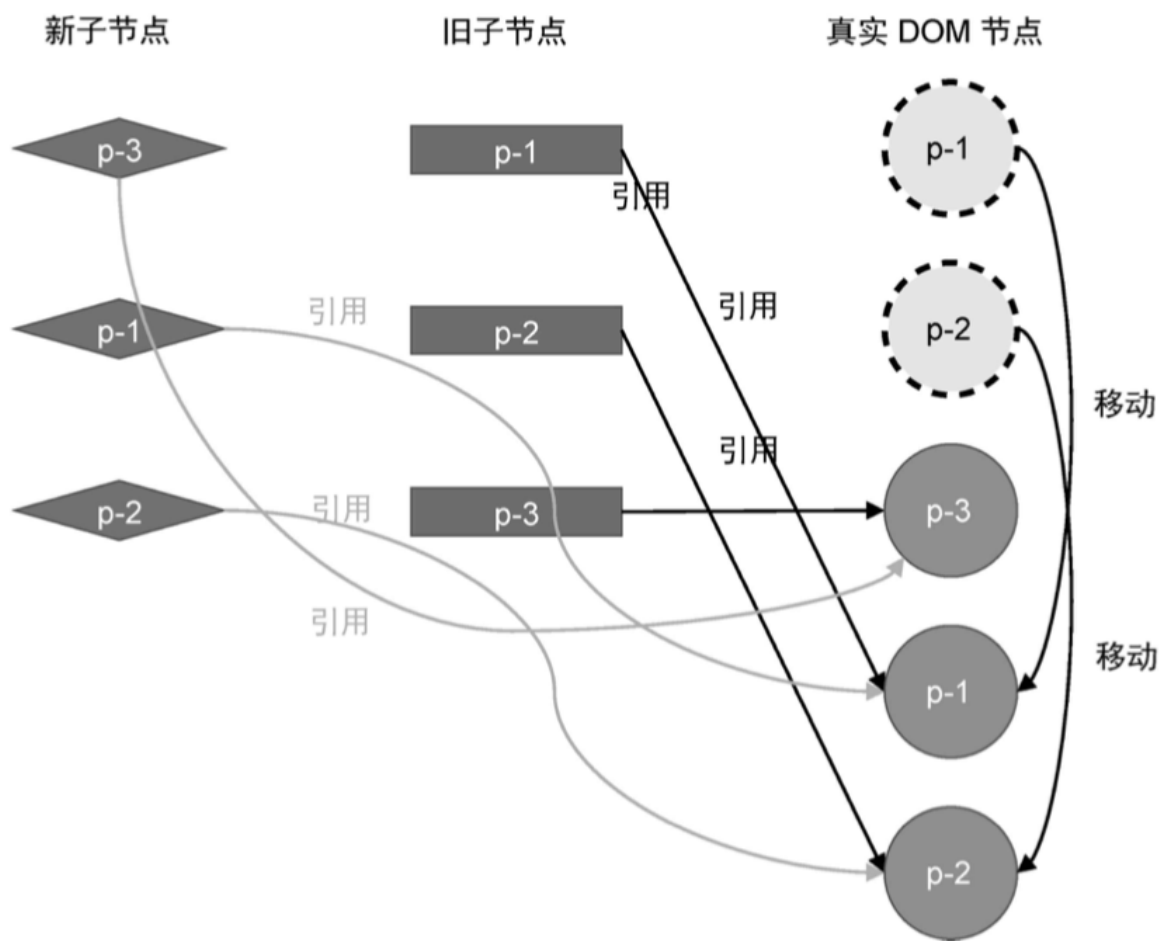
移动

- 第一步：取新的一组子节点中第一个节点 p-3，它的 key 为 3，尝试在旧的一组子节点中找到具有相同 key 值的可复用节点。发现能够找到，并且该节点在旧的一组子节点中的索引为 2。此时变量 lastIndex 的值为 0，索引 2 不小于 0，所以节点 p-3 对应的真实 DOM 不需要移动，但需要更新变量 lastIndex 的值为 2。

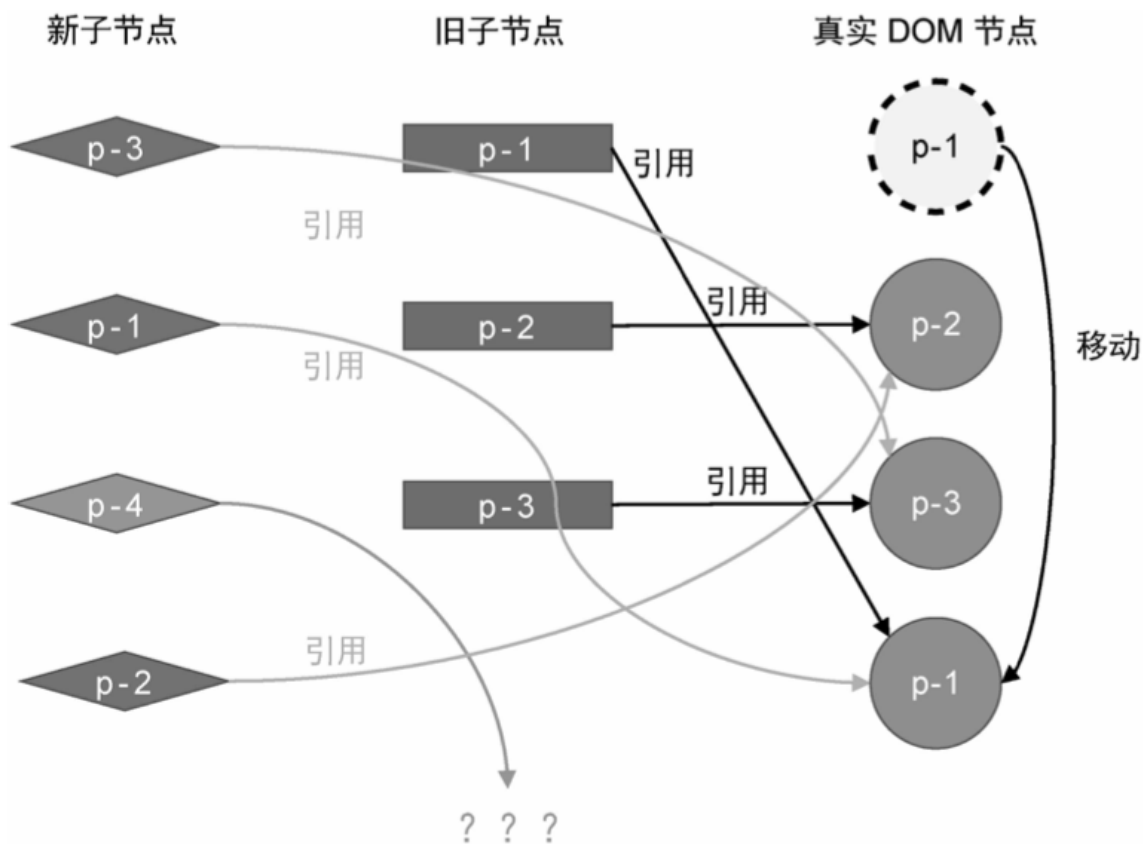
- 第二步：取新的一组子节点中第二个节点 p-1，它的 key 为 1，尝试在旧的一组子节点中找到具有相同key值的可复用节点。发现能够找到，并且该节点在旧日的一组子节点中的索引为 0。此时变量 lastIndex 的值为 2，索引 0 小于2，所以节点 p-1 对应的真实 DOM 需要移动。
到了这一步，我们发现，节点 p-1 对应的真实 DOM 需要移动，但应该移动到哪里呢？我们知道，新 children 的顺序其实就是更新后真实 DOM 节点应有的顺序。所以节点 p-1 在新 children 中的位置就代表了真实 DOM 更新后的位置。由于节点 p-1 在新children 中排在节点p-3 后面，所以我们应该把节点 p-1 所对应的真实 DOM 移动到节点p-3所对应的真实 DOM 后面。
可以看到，这样操作之后，此时真实 DOM 的顺序为 **p-2、p-3、p-1**。

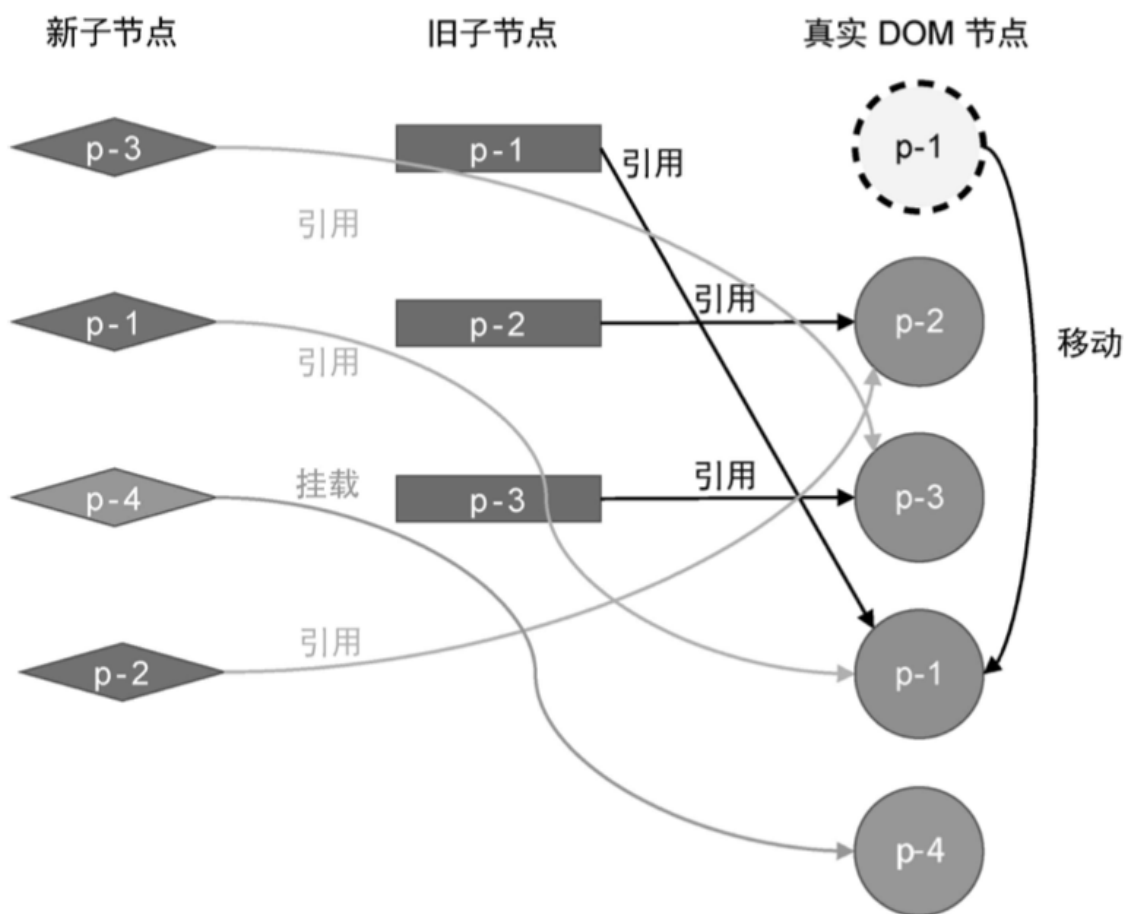


- 第三步：取新的一组子节点中第三个节点 p-2，它的 key 为 2。
尝试在旧的一组子节点中找到具有相同 key 值的可复用节点。发现能够找到，并且该节点在旧的一组子节点中的索引为 1。此时变量 lastIndex 的值为 2，索引 1 小于 2，所以节点 p-2 对应的真实 DOM 需要移动。原理与第二步相似。



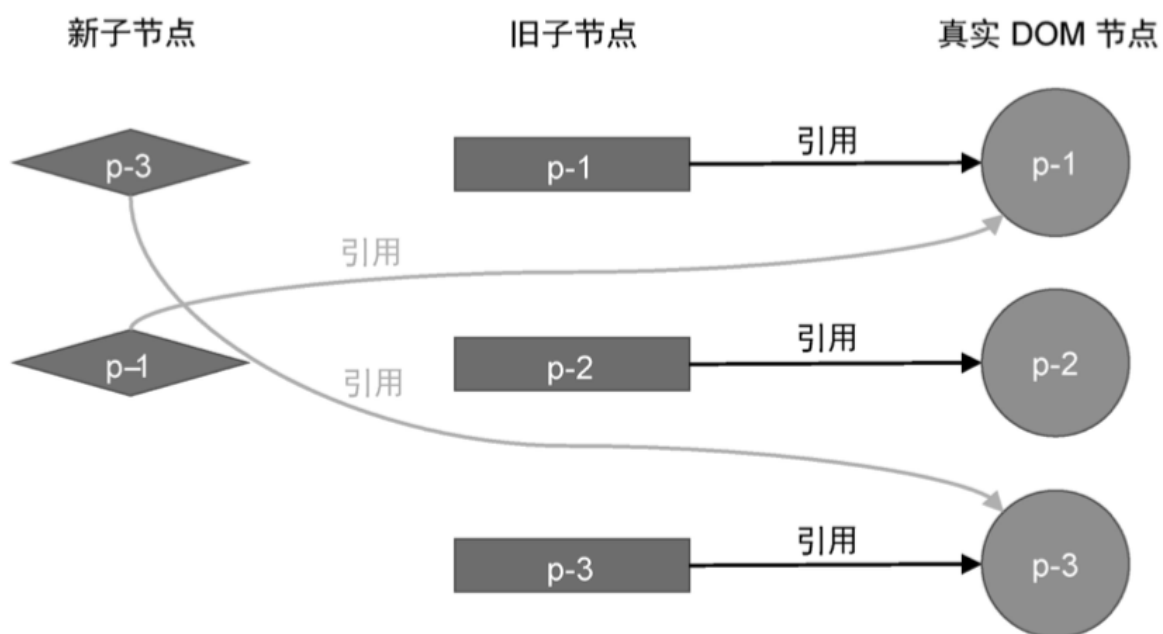
新增





两次移动，一次新增

删除



一次移动，一次删除。

节点 p-2 对应的真实 DOM 仍然存在，所以需要增加额外的逻辑来删除遗留节点。思路很简单，当基本的更新结束时，我们需要遍历旧的一组子节点，然后去新的一组子节点中寻找具有相同 key 值的节点。如果找不到，则说明应该删除该节点。

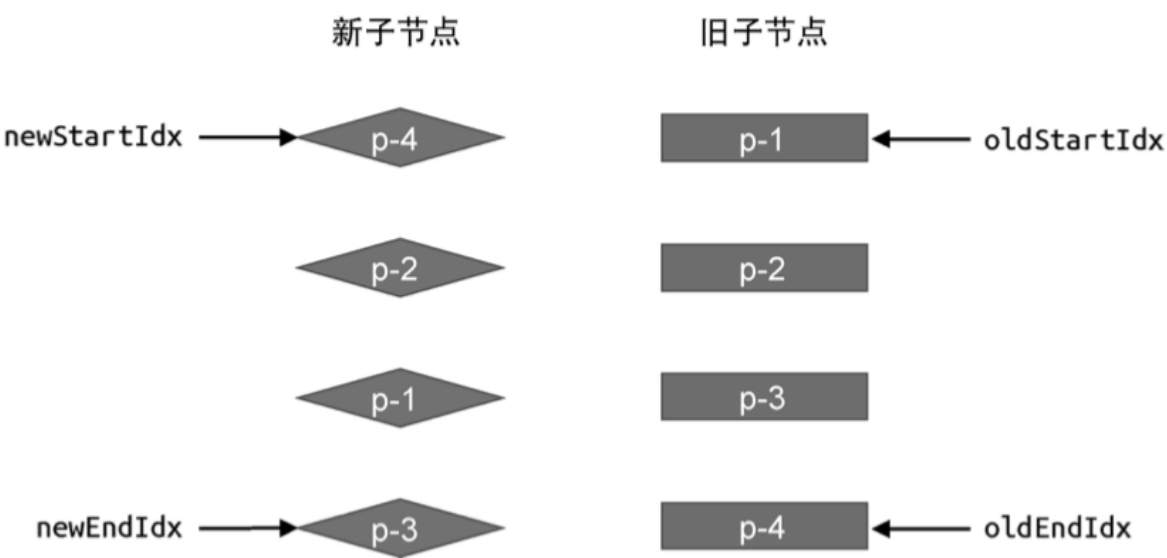
<https://github.com/vuejs/vue/blob/49b6bd4264c25ea41408f066a1835f38bf6fe9f1/src/core/vdom/patch.ts#L413>

简单 Diff 算法的核心逻辑是，拿新的一组子节点中的节点去旧的一组子节点中寻找可复用的节点。如果找到了，则记录该节点的位置索引。我们把这个位置索引称为最大索引。在整个更新过程中，如果一个节点的索引值小于最大索引，则说明该节点对应的真实 DOM 元素需要移动。

最后我们整体介绍了渲染器如何移动、添加、删除 虚拟节点所对应的 DOM 元素。

双端 diff

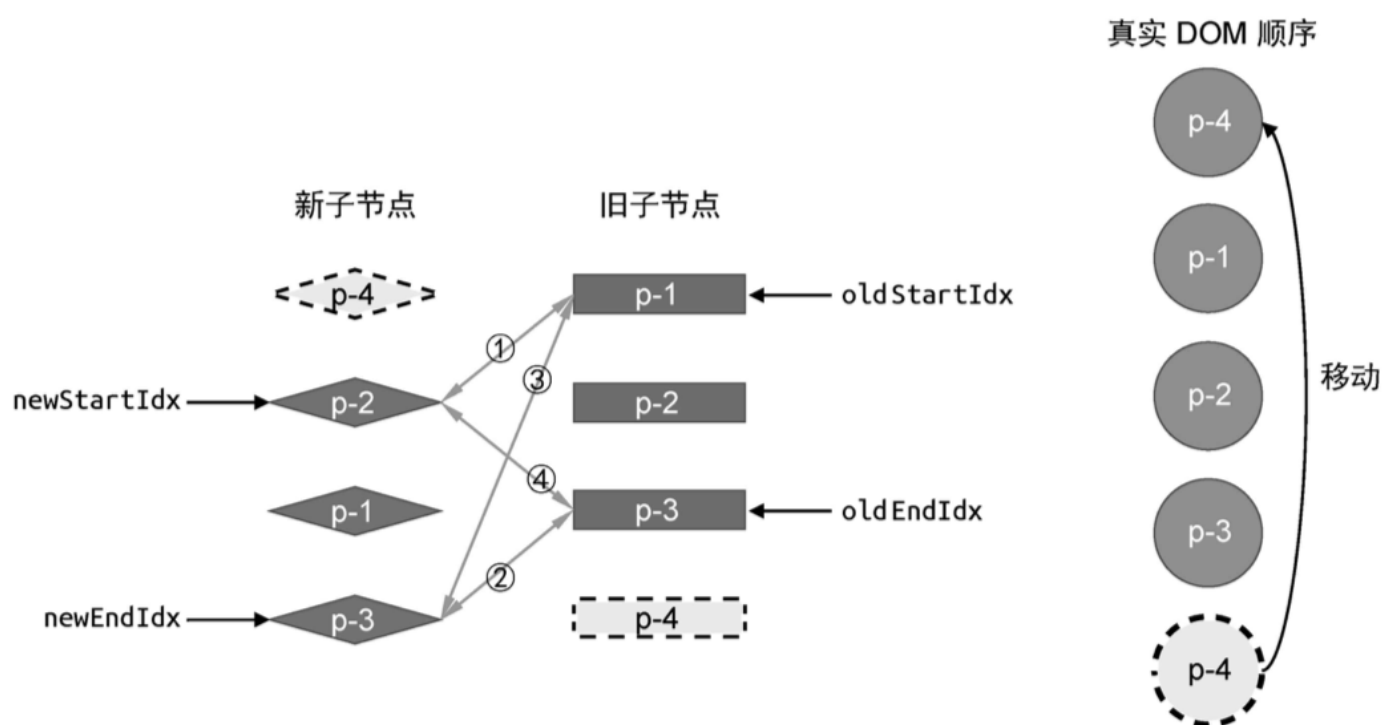
双端 diff 顾名思义，对比的逻辑分别从两端收敛进行。双端 Diff 算法是一种同时对新旧两组子节点的两个端点进行比较的算法。因此，我们需要四个索引值，分别指向新旧两组子节点的端点。



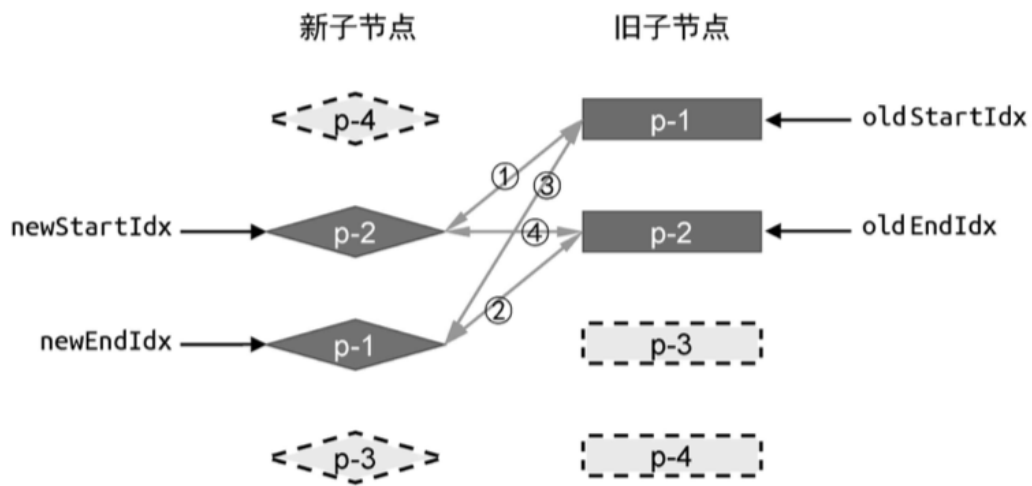
- 首首比较
- 尾尾比较
- 尾首比较
- 首尾比较

详述一下执行步骤：

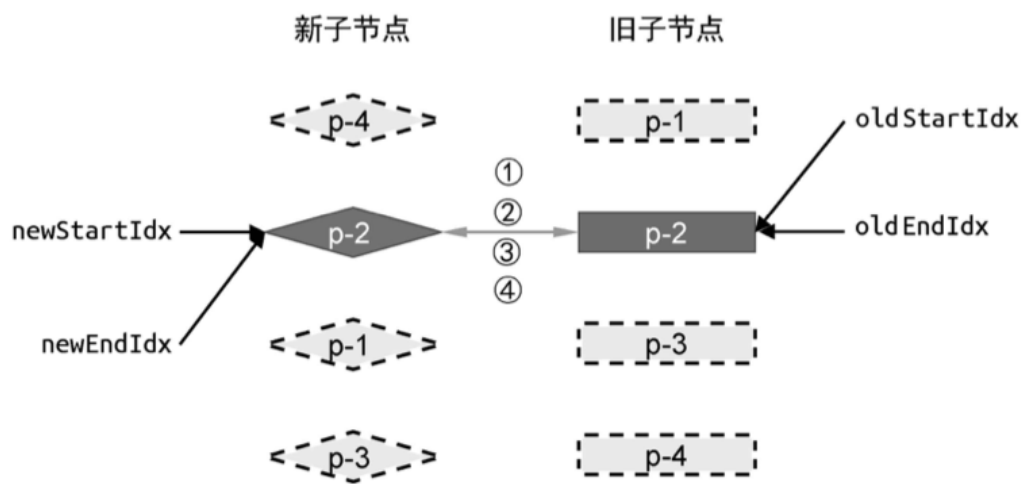
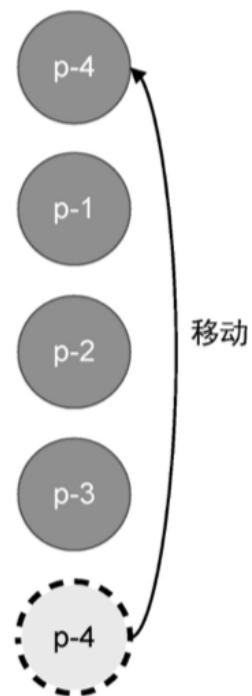
- 第一步：比较旧的一组子节点中的第一个子节点 p-1 与新的一组子节点中的第一个子节点 p-4，看看它们是否相同。由于两者的 key 值不同，因此不相同，不可复用，于是什么都不做。
- 第二步：比较旧的一组子节点中的最后一个子节点 p-4 与新的一组子节点中的最后一个子节点 p-3，看看它们是否相同。由于两者的 key 值不同，因此不相同，不可复用，于是什么都不做。
- 第三步：比较旧的一组子节点中的第一个子节点 p-1 与新的一组子节点中的最后一个子节点 p-3，看看它们是否相同。由于两者的 key 值不同，因此不相同，不可复用，于是什么都不做。
- 第四步：比较旧的一组子节点中的最后一个子节点 p-4 与新的一组子节点中的第一个子节点 p-4。由于它们的 key 值相同，因此可以进行 DOM 复用。



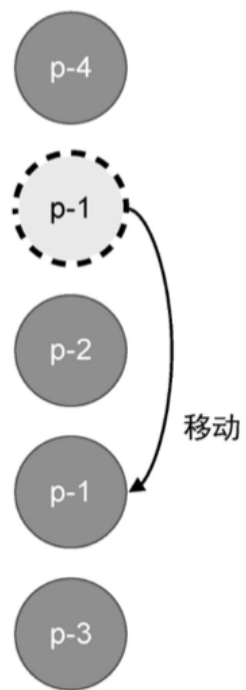
对比完以后，接下来反复如此，直到不满足 `oldStartIdx <= oldEndIdx && newStartIdx <= newEndIdx` 这一条件

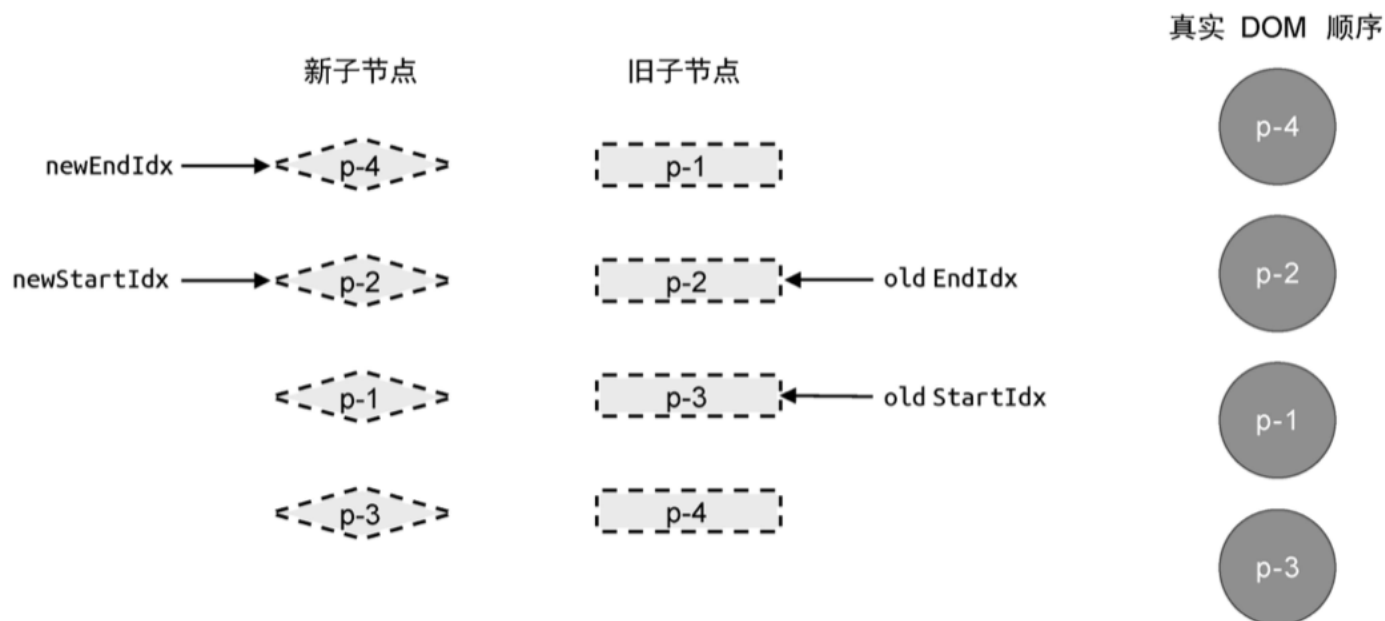


真实 DOM 顺序



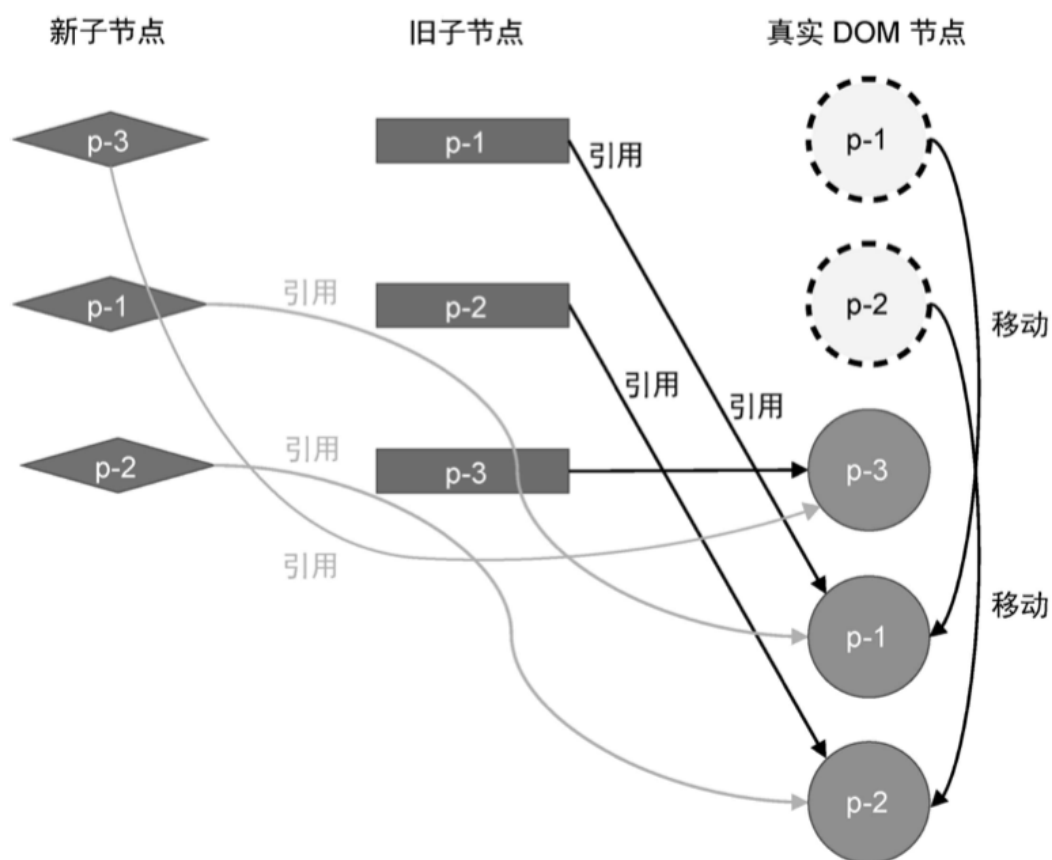
真实 DOM 顺序





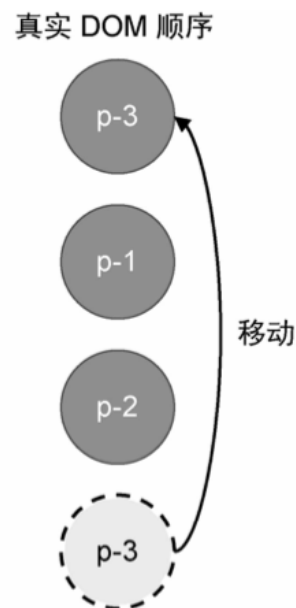
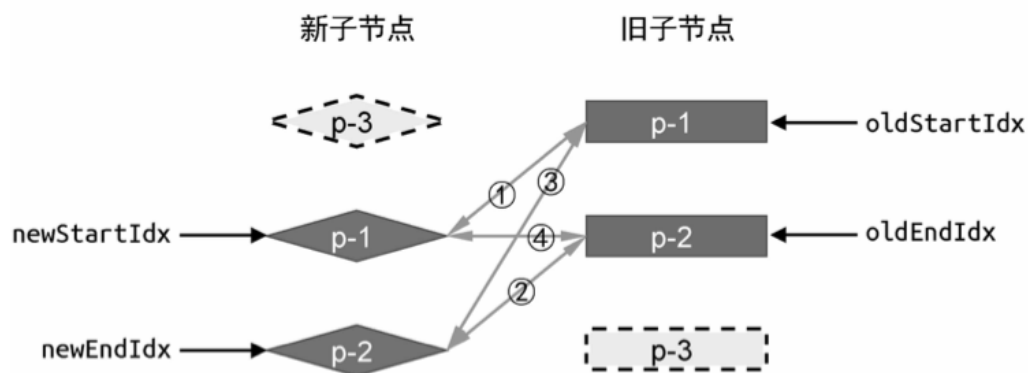
双端 diff vs 简单 diff

简单 diff



我们发现，这样的一次 diff，共需要进行两次 dom 移动。

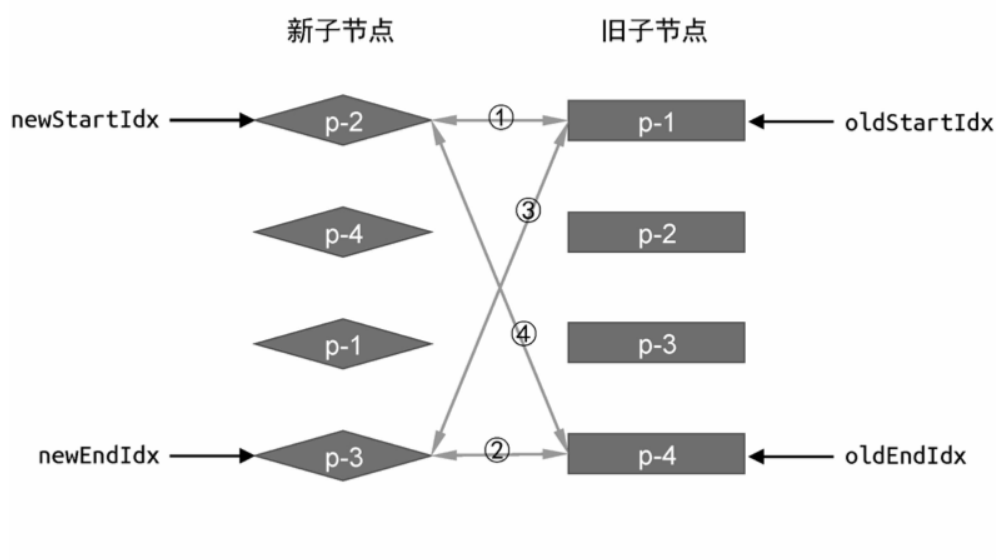
双端 diff



而双端 diff 只需要一次。

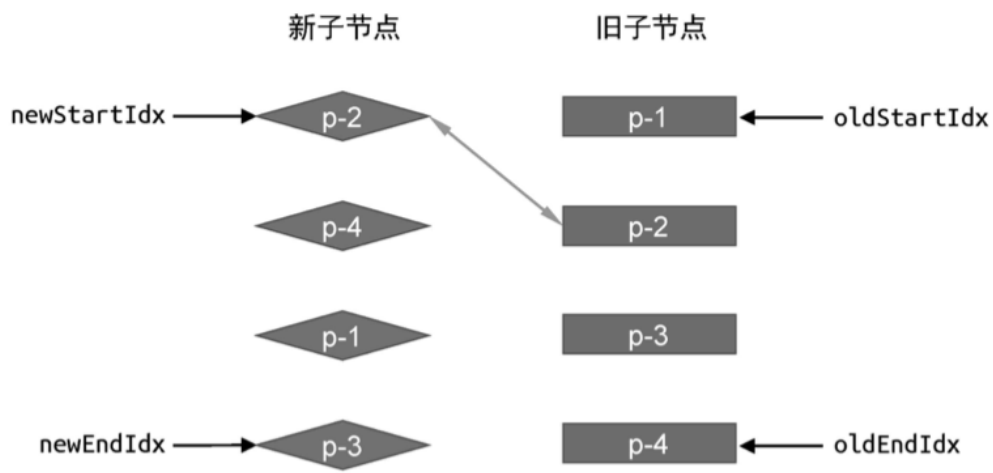
双端 diff 的非理想情况处理

大家想想，如果一轮对比发现根本就没有可以复用的内容，该怎么处理？

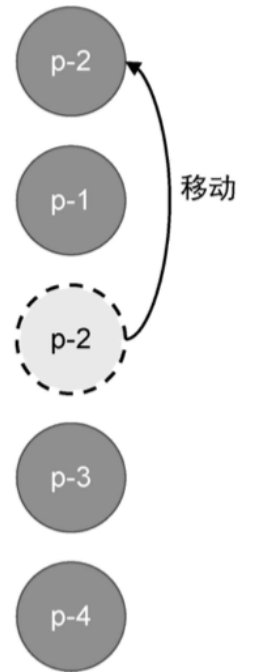


我们从上图可以发现，第一轮对比都没有命中可复用节点，怎么办呢？

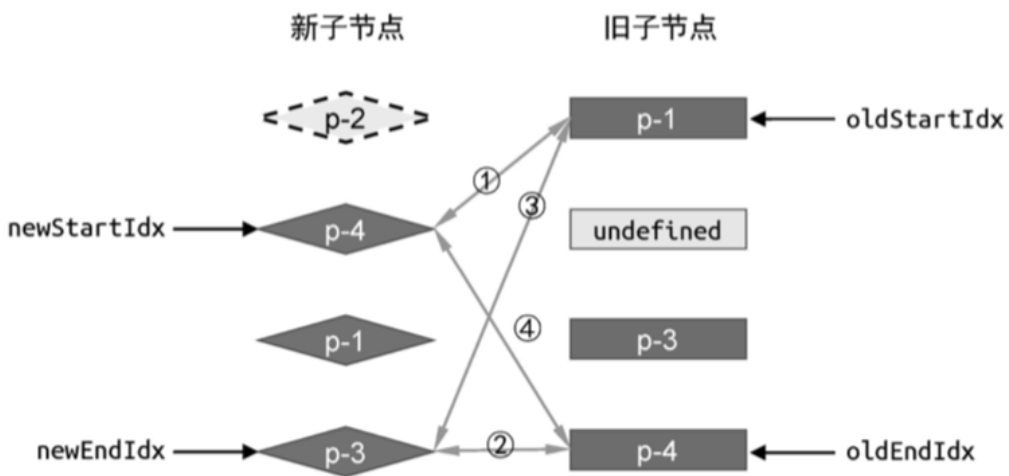
我们就只能拿新子节点中第一个节点 p-2 去旧子节点中找



真实 DOM 顺序



找到啦，那就移动，且将旧子节点 p-2 赋为 undefined。

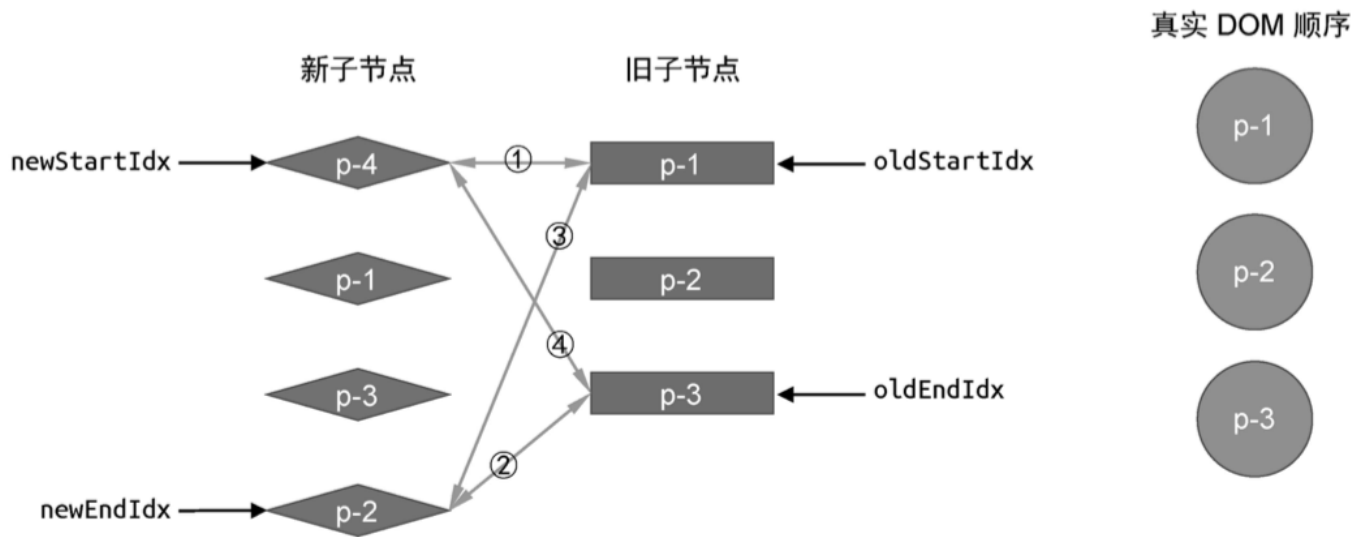


真实 DOM 顺序

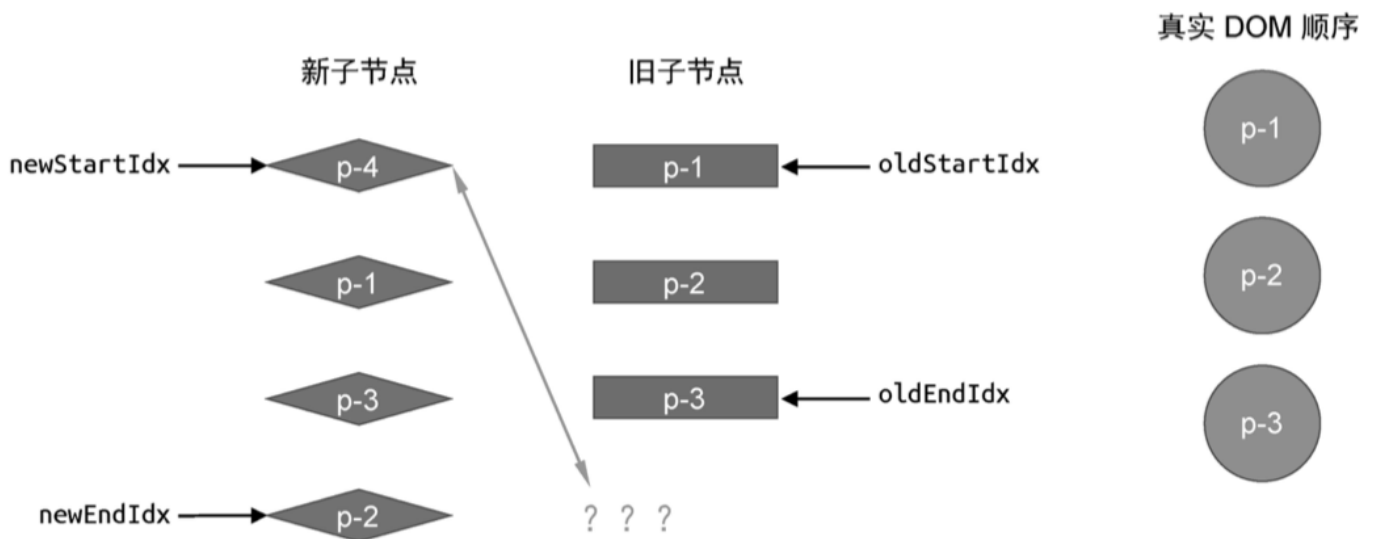


以此，完成后续比对。

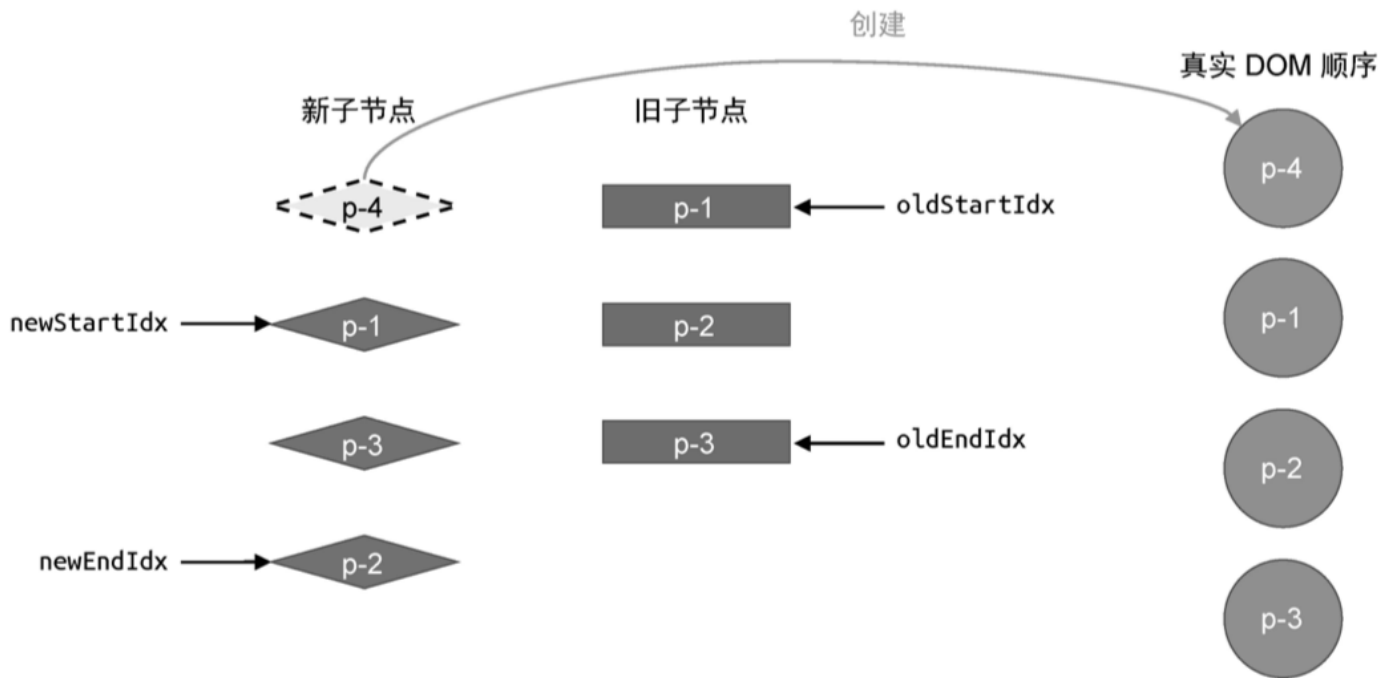
添加新元素



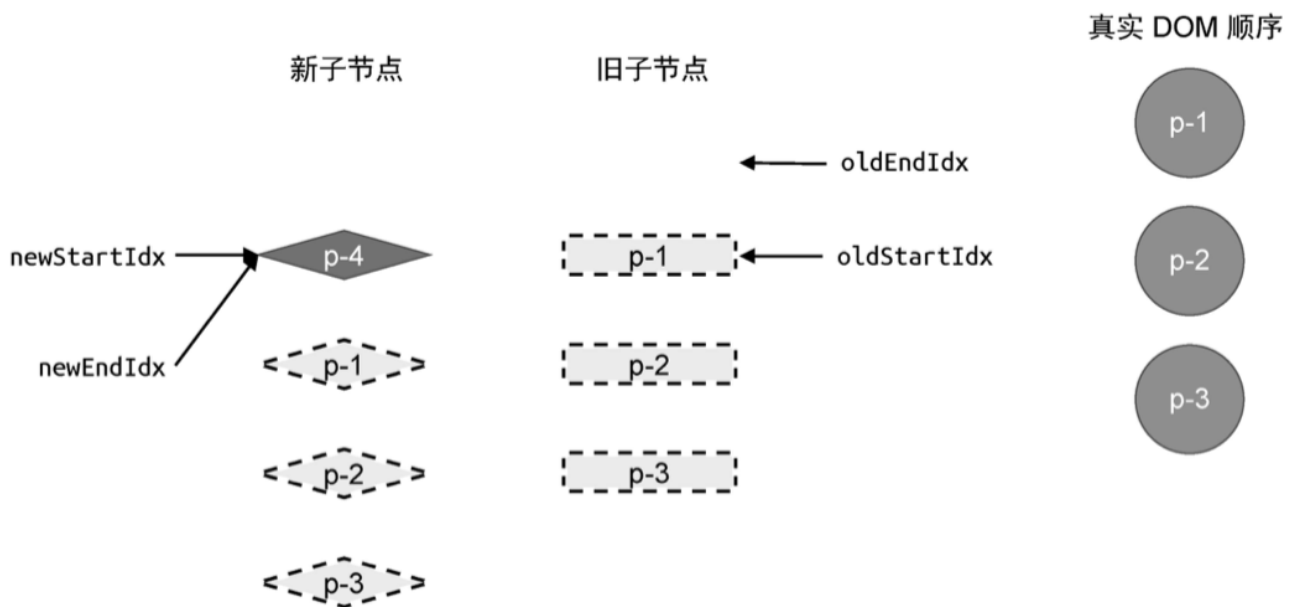
对比完，发现，糟糕，没有任何可复用的节点，那么接下来尝试拿出新节点中第一个节点去旧节点找



新子节点第一个节点 p-4 在旧子节点也没有找到，情况不妙，说明这个 p-4 是新成员，那么我们就需要创建 dom，然后 newStartIdx 后移。



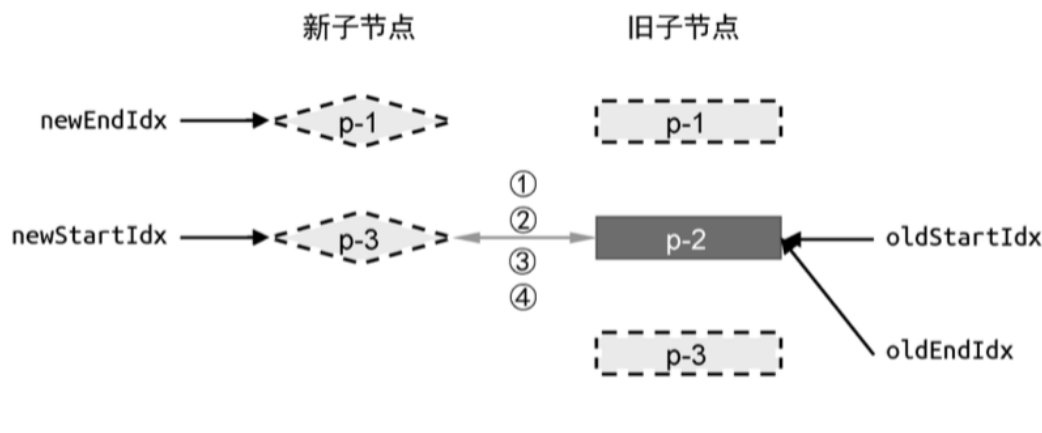
还有一种情况，就是比对时，尾部有可复用元素，直到新子节点前端发现无法复用。



删除元素

删除元素其实很简单，就是在 diff 过程完成后，如果 $\text{newEndIdx} < \text{newStartIdx}$ 了，但是此时旧子节点的 $\text{oldStartIdx} \leq \text{oldEndIdx}$ ，则需要将 $\text{oldStartIdx} \sim \text{oldEndIdx}$ 的所有元素删除。

真实 DOM 顺序



快速 diff（重点关注）

快速 diff 这一算法借鉴了 ivi 或者 inferno 这两个库的实现。

<https://github.com/infernojs/inferno/blob/a59d1fbe296b9468823a59f34f6591d8591d2759/packages/inferno/src/DOM/patching.ts#L1223>

最长递增子序列

这是快速 diff 算法的核心，我们非常有必要将其单独抽取出来进行讲解。

对应 LeetCode，<https://leetcode.cn/problems/longest-increasing-subsequence/description/>

比如我现在有一个数组，[4,6,7]，那么它的递增子序列有以下几个：

- [4,6]
- [4,6,7]
- [4,7]
- [6,7]
- [7]

那么可以看出，最长递增子序列是 [4,6,7]

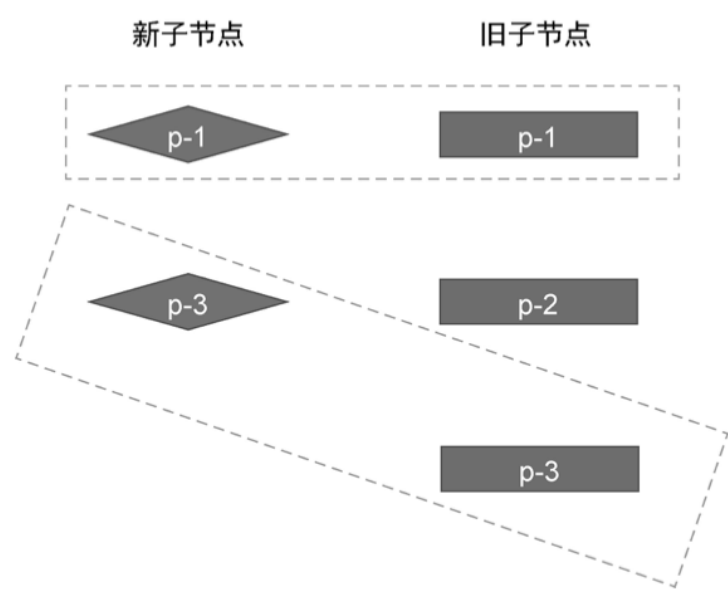
预处理

Vue3 在 diff 时会预先进行优化处理，怎么做呢？我们可以看看如下示例：

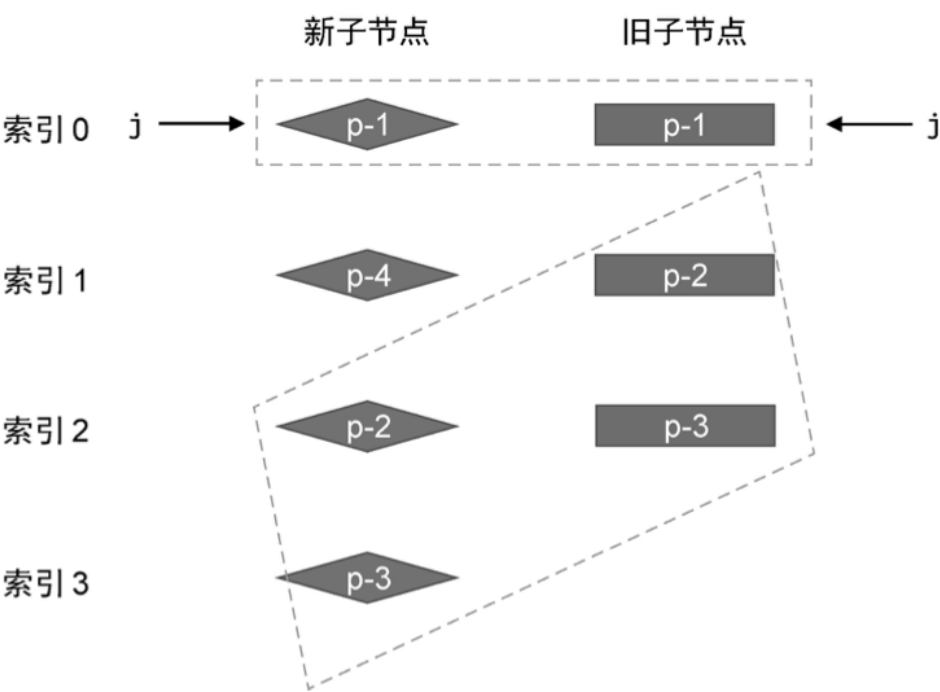
```
1  const text1 = 'Hello World'
2  const text2 = 'Hello'
```

那其实，我们真正需要 diff 的只有 'World'，为什么，因为字符串前后我们可以先剔除掉相同子串。

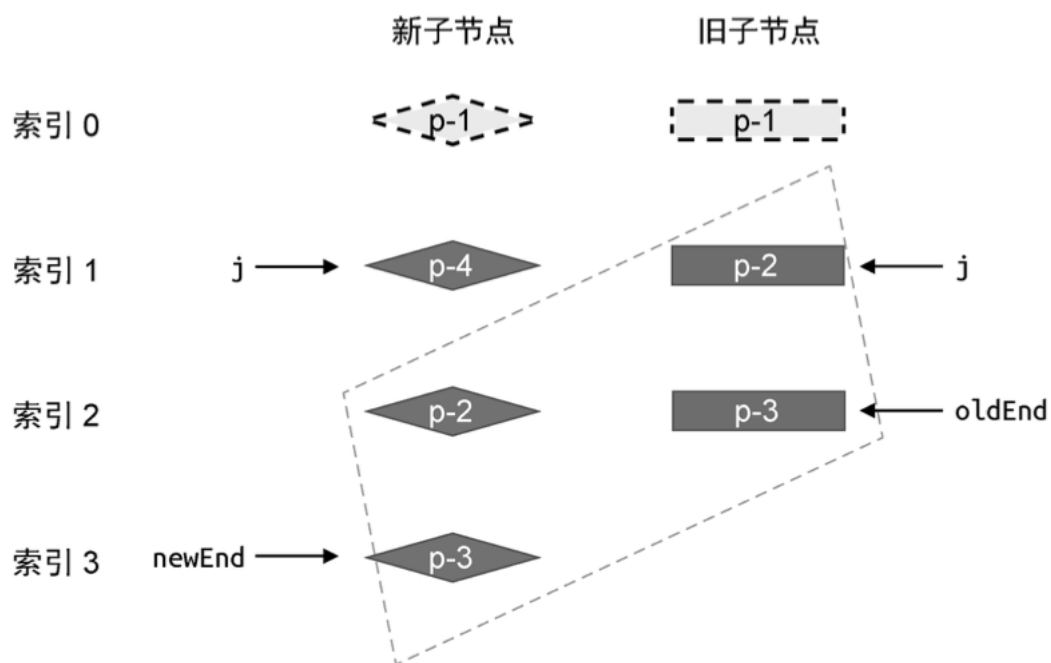
那在 Vue diff 时，也一样



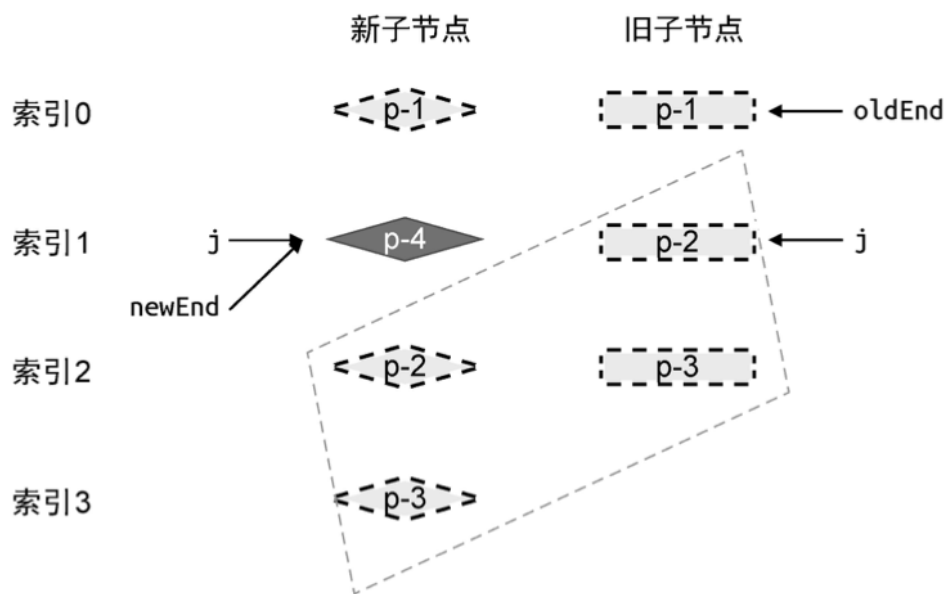
我们先将首尾相同节点 diff 并在发现相同元素时进行 patch 操作。



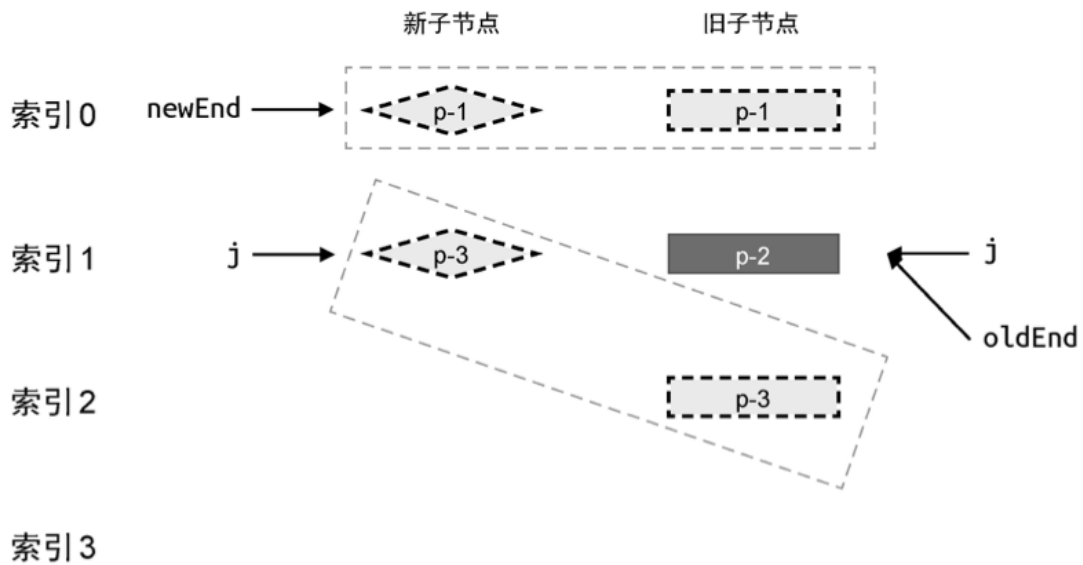
先定义一个索引 j，逐步递增，对比新旧子节点如果相同则进行 patch 操作，直至不相同，当不相同
时，我们转变处理方法，从尾部开始进行对比，如下：



很显然，这里执行完以后，只剩下 p-4，所以这个作为新节点进行挂载，需要创建新 dom。



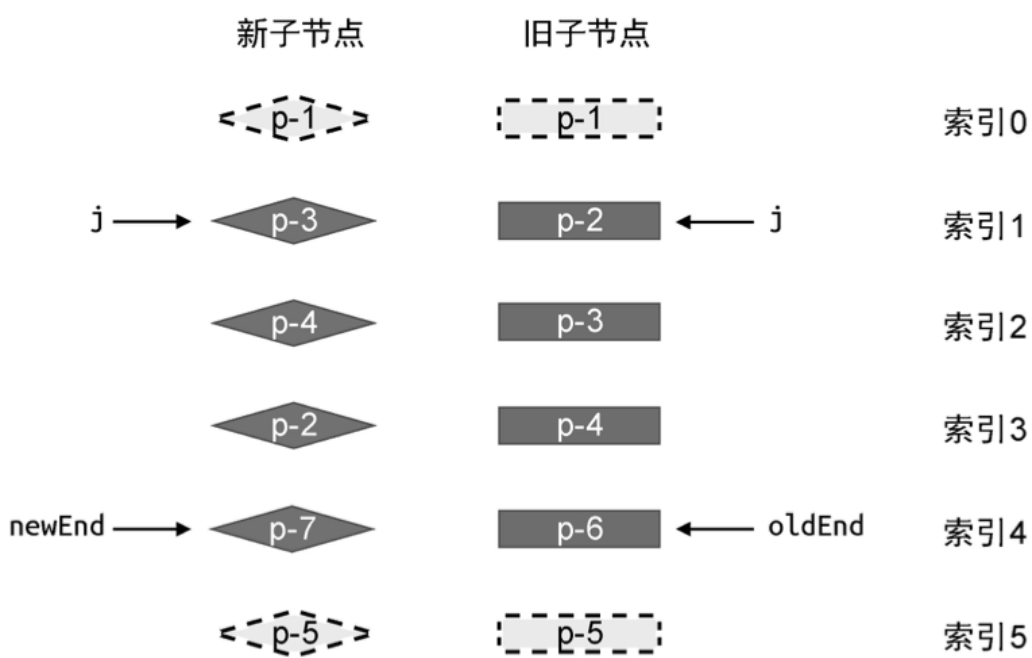
还存在另外一种情况，就是最终剩下旧子节点，那么旧子节点对应 dom 会被移除。如图：



总结下来就是：

- 遍历完后，如果新子节点 $\text{newEnd} > j$ ，则 $\text{newEnd} \sim j$ 的子元素全部作为新元素挂载，需创建 dom
- 遍历完后，如果旧子节点 $j > \text{oldEnd}$ ，则 $j \sim \text{oldEnd}$ 的子元素全部移除，需删除 dom

DOM 移动



接下来构造 source 数组，这就是为我们后面计算**最长递增子序列**做准备的。

初始化的逻辑简化代码如下：

```

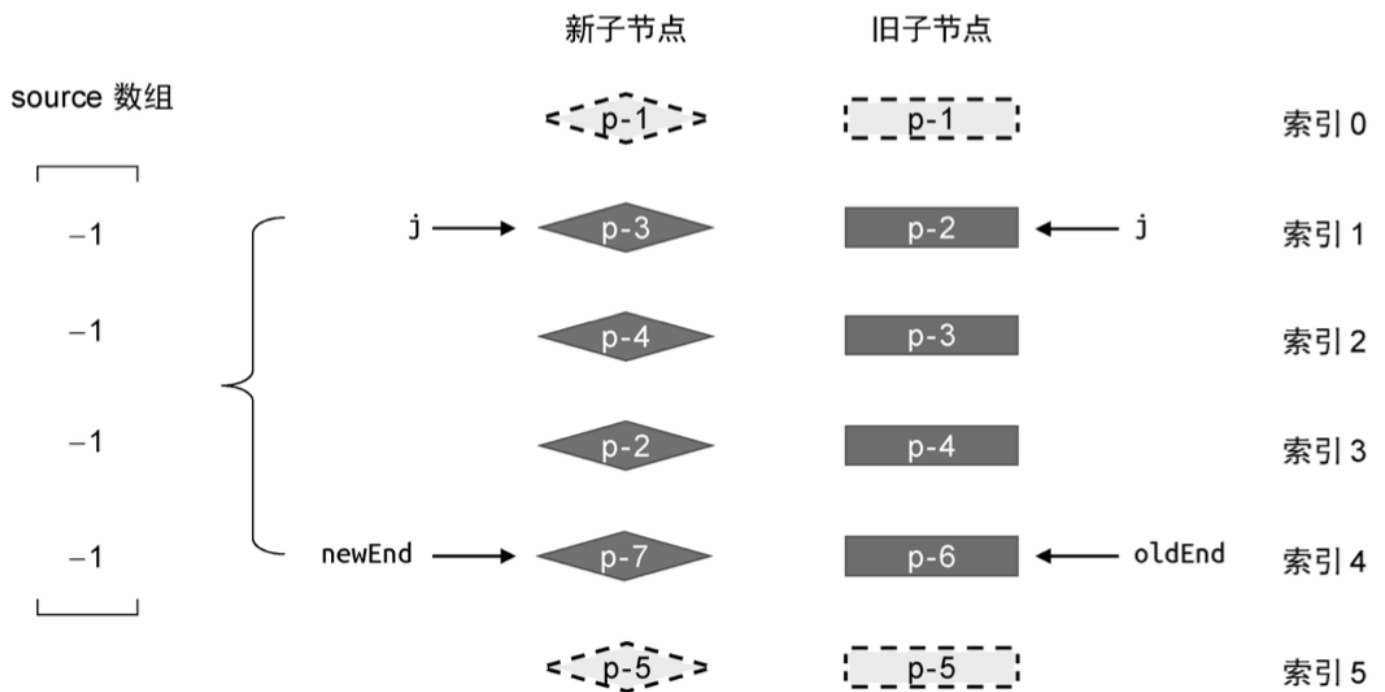
1  const count = newEnd - j + 1;
2  const source = new Array(count);
3  source.fill(-1)

```

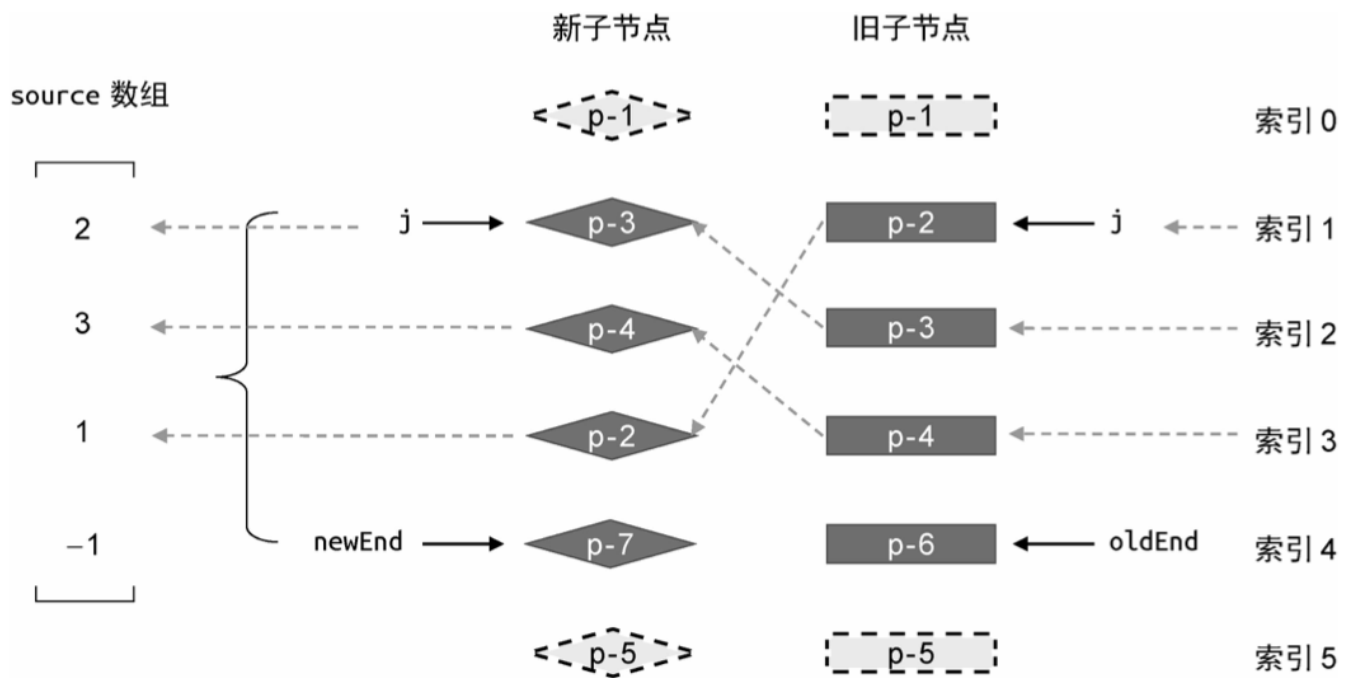
```

4
5 // 3.3.4
6 // core-main/packages/runtime-core/src/renderer.ts 1905 行
7 const newIndexToOldIndexMap = new Array(toBePatched)
8 for (i = 0; i < toBePatched; i++) newIndexToOldIndexMap[i] = 0

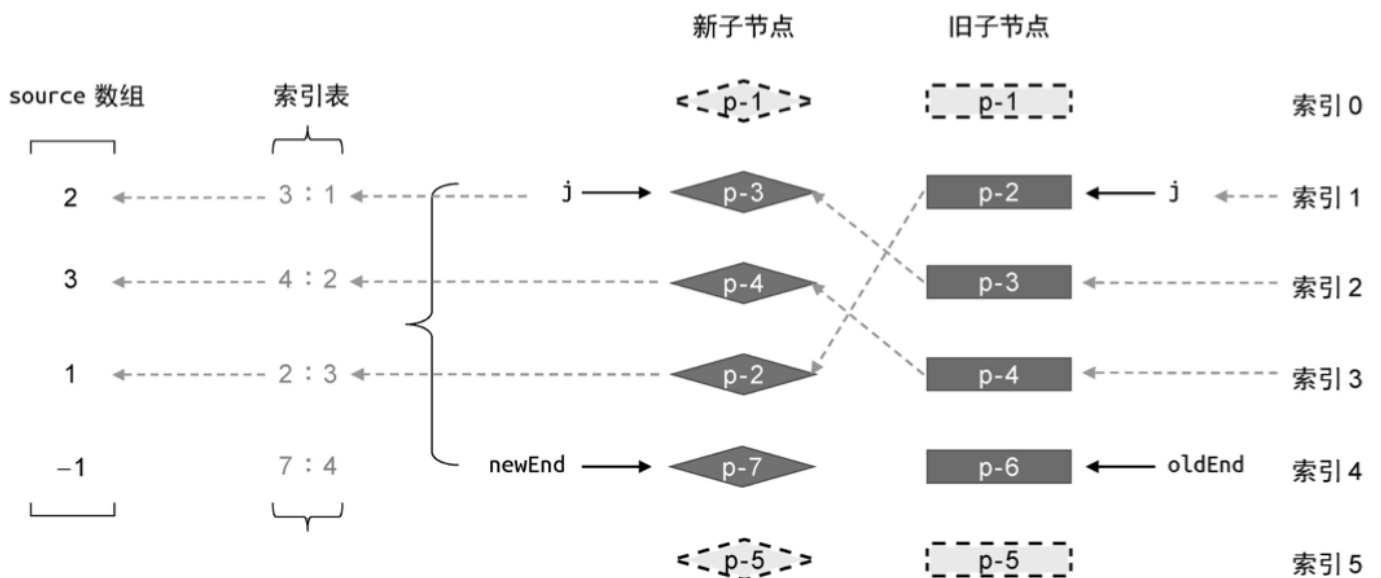
```



然后将每一位的值设置为该节点在旧子节点中的索引，填充后，source 数组更新为：`[2,3,1,-1]`，其中 -1 表示在旧子节点中没有对应节点。



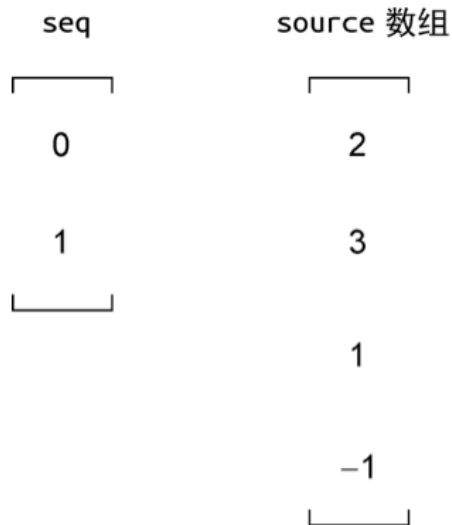
接下来为了性能优化考虑，需要额外新建一张索引表，用于标志新旧子节点的对照关系，构建为：



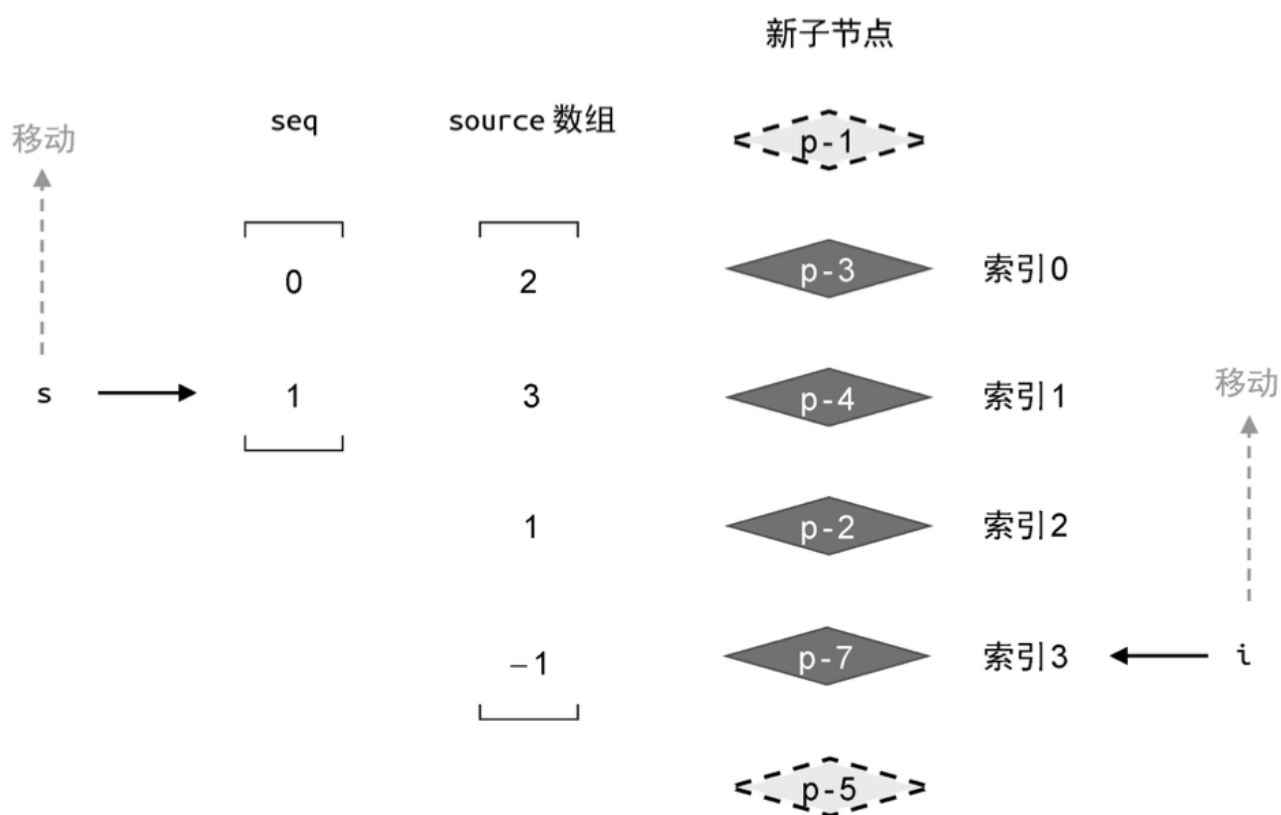
我们来看看，这个 source 数组的最长递增子序列是？考下大家 [2, 3, 1, -1]

答案是： [2, 3]

但是我们会发现，在 Vue 源码中并不是，Vue 源码计算出的结果是 [0, 1]，为什么呢？因为我们需要得到的是索引，因为索引才是后续我们该如何移动的关键。



接下来我们重点关注需要更新的节点序列，并重新编号



对比思路如下：i--

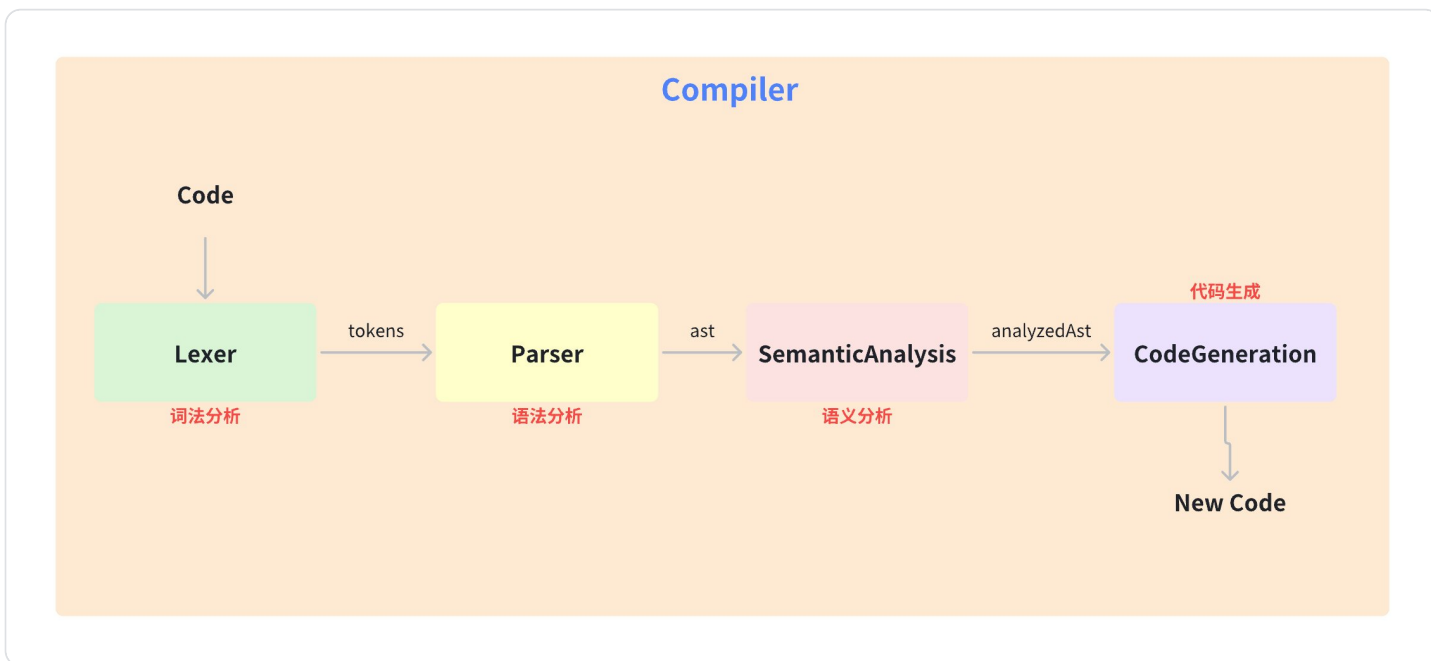
1. 首先看 source[i] 是否为 -1，如果是则表示需要**新增 dom**，否则走下一步判断逻辑
2. 再看 seq[s] 是否等于 i，如果是，则不需要处理，否则需要**移动 dom**
3. 如果 seq[s] 等于 i，则 s--

直至结束

Vue3 diff 优化

1. **静态标记 + 非全量 Diff**：（Vue 3在创建虚拟DOM树的时候，会根据DOM中的内容会不会发生变化，添加一个静态标记。之后在与上次虚拟节点进行对比的时候，就只会对比这些带有静态标记的节点。）；
2. 使用最长递增子序列（快速 diff）优化对比流程，可以最大程度的减少 DOM 的移动，达到最少的 DOM 操作；

6. 编译器



- Parser
- Transformer
- Generator

以上是编译器基本原理，在 Vue 中，我们需要实现模板到函数的转换，同理在 react 中，我们需要实现 jsx 到 React.createElement 的转换。

```
1 <div>
2   <p>Vue</p>
3   <p>Template</p>
4 </div>
```

假设我们有如上模板，很显然我们函数是无法使用它的，我们必须把它转换为函数 `h` 形式，如下：

```
1 function render() {
```

```
2     return h('div', [  
3         h('p', 'Vue'),  
4         h('p', 'Template')  
5     ])  
6 }
```

从零实现

parser

首先我们实现 parser，将模板转为初始 ast

```
1  // 定义状态机的状态  
2  const State = {  
3      initial: 1, // 初始状态  
4      tagOpen: 2, // 标签开始状态  
5      tagName: 3, // 标签名称状态  
6      text: 4, // 文本状态  
7      tagEnd: 5, // 结束标签状态  
8      tagEndName: 6, // 结束标签名称状态  
9  }  
10  
11 // 辅助函数，用于判断是否是字母  
12 function isAlpha(char) {  
13     return char >= 'a' && char <= 'z' || char >= 'A' && char <= 'Z'  
14 }  
15  
16 // 接收模板字符串作为参数，并将模板切割为 Token 返回  
17 function tokenize(str) {  
18     // 状态机的当前状态：初始状态  
19     let currentState = State.initial  
20     // 缓存字符  
21     const chars = []  
22     // 生成的 Token 会存储到 tokens 数组中，并作为函数的返回值返回  
23     const tokens = []  
24  
25     // 使用 while 循环开启自动机  
26     while (str) {  
27         const char = str[0]  
28  
29         // switch 匹配当前状态  
30         switch (currentState) {  
31             case State.initial: // 初始状态
```

```
32 // 遇到字符 <
33 if (char === '<') {
34     // 1. 状态机切换到标签开启状态
35     currentState = State.tagOpen
36     // 2. 消费字符 <
37     str = str.slice(1)
38 } else if (isAlpha(char)) {
39     // 1. 遇到字母, 切换到文本状态
40     currentState = State.text
41     // 2. 将当前字母缓存到 chars 数组
42     chars.push(char)
43     // 3. 消费当前字符
44     str = str.slice(1)
45 }
46 break;
47 case State.tagOpen: // 标签开始状态
48     if (isAlpha(char)) {
49         // 1. 遇到字母, 切换到标签名称状态
50         currentState = State.tagName
51         // 2. 将当前字符缓存到 chars 数组
52         chars.push(char)
53         // 3. 消费当前字符
54         str = str.slice(1)
55     } else if (char === '/') {
56         // 1. 遇到字符 /, 切换到结束标签状态
57         currentState = State.tagEnd
58         // 2. 消费字符 /
59         str = str.slice(1)
60     }
61     break;
62 case State.tagName: // 标签名称状态
63     if (isAlpha(char)) {
64         // 1. 遇到字母, 保持状态不变, 缓存当前字符到 chars 数组
65         chars.push(char)
66         // 2. 消费当前字符
67         str = str.slice(1)
68     } else if (char === '>') {
69         // 1. 遇到字符 >, 切换到初始状态
70         currentState = State.initial
71         // 2. 创建一个标签 Token, 并添加到 tokens 数组中
72         // tip: chars 数组中缓存的字符就是标签名称
73         tokens.push({
74             type: 'tag',
75             name: chars.join('')
76         })
77         // 3. 清空已消费的 chars 数组
78         chars.length = 0
```

```

79         // 4. 消费当前字符 >
80         str = str.slice(1)
81     }
82     break;
83 case State.text: // 文本状态
84     if (isAlpha(char)) {
85         // 1. 遇到字母, 保持状态不变, 缓存当前字符到 chars 数组
86         chars.push(char)
87         // 2. 消费当前字符
88         str = str.slice(1)
89     } else if (char === '<') {
90         // 1. 遇到字符 <, 切换到标签开始状态
91         currentState = State.tagOpen
92         // 2. 从文本状态 --> 标签开始状态, 此时应该创建文本 Token, 并添加到 Token
数组中
93         // tip: cahrs 数组中缓存的字符就是文本内容
94         tokens.push({
95             type: 'text',
96             content: chars.join('')
97         })
98         // 3. 清空已消费的 chars 数组
99         chars.length = 0
100        // 4. 消费当前字符 <
101        str = str.slice(1)
102    }
103    break;
104 case State.tagEnd: // 标签结束状态
105     if (isAlpha(char)) {
106         // 1. 遇到字母, 切换到结束标签名称状态
107         currentState = State.tagEndName
108         // 2. 将当前字符缓存到 chars 数组
109         chars.push(char)
110         // 3. 消费当前字符
111         str = str.slice(1)
112     }
113     break;
114 case State.tagEndName: // 结束表明名称状态
115     if (isAlpha(char)) {
116         // 1. 遇到字母, 保持状态不变, 缓存当前字符到 chars 数组
117         chars.push(char)
118         // 2. 消费当前字符
119         str = str.slice(1)
120     } else if (char === '>') {
121         // 1. 遇到字母 >, 切换到初始状态
122         currentState = State.initial
123         // 2. 从结束标签名称状态 --> 初始状态, 应该保存结束标签名称 Token
124         // tip: cahrs 数组中缓存的字符就是标签名称

```

```

125         tokens.push({
126             type: 'tagEnd',
127             name: chars.join('')
128         })
129         // 3. 清空已消费的 chars 数组
130         chars.length = 0
131         // 4. 消费当前字符 >
132         str = str.slice(1)
133     }
134     break;
135 }
136 }
137
138 // 返回 tokens
139 return tokens
140 }
141
142 // parse 函数接收模板作为参数
143 function parse(str) {
144     // 首先对模板进行标记化, 得到 tokens
145     const tokens = tokenize(str)
146     // 创建 Root 根节点
147     const root = {
148         type: 'Root',
149         children: []
150     }
151     // 创建 elementStack 栈
152     const elementStack = [root]
153
154     // 开启 while 循环扫描 tokens
155     while (tokens.length) {
156         // 获取当前栈顶节点作为父节点 parent
157         const parent = elementStack[elementStack.length - 1]
158         // 当前扫描的 Token
159         const t = tokens[0]
160
161         switch (t.type) {
162             case 'tag':
163                 // 如果当前 Token 是开始标签, 创建 Element 类型的 AST 节点
164                 const elementNode = {
165                     type: 'Element',
166                     tag: t.name,
167                     children: []
168                 }
169                 // 将其添加到父级节点的 children 中
170                 parent.children.push(elementNode)
171                 // 将当前节点压入栈

```

```

172     elementStack.push(elementNode)
173     break;
174   case 'text':
175     // 如果当前 Token 是文本, 创建 Text 类型的 AST 节点
176     const textNode = {
177       type: 'Text',
178       content: t.content
179     }
180     // 将其添加到父节点的 children 中
181     parent.children.push(textNode)
182     break;
183   case 'tagEnd':
184     // 遇到结束标签, 将栈顶节点弹出
185     elementStack.pop()
186     break;
187   }
188
189   // 消费已扫描过的 token
190   tokens.shift()
191 }
192
193 // 返回 ast
194 return root
195 }
196
197 module.exports = {
198   parse
199 }

```

transformer

```

1  function dump(node, indent = 0) {
2    // 节点类型
3    const type = node.type
4    // 节点的描述, 如果是根节点, 则没有描述
5    // 如果是 Element 类型的节点, 则使用 node.tag 作为节点的描述
6    // 如果是 Text 类型的节点, 则使用 node.content 作为节点的描述
7    const desc = node.type === 'Root'
8      ? ''
9      : node.type === 'Element'
10       ? node.tag
11       : node.content
12    // 打印节点的类型和描述信息
13    console.log(`${' '.repeat(indent)}${type}: ${desc}`)
14    // 递归地打印子节点

```

```
15     if (node.children) {
16         node.children.forEach(n => dump(n, indent + 2))
17     }
18 }
19
20 // 用来创建 StringLiteral 节点
21 function createStringLiteral(value) {
22     return {
23         type: 'StringLiteral',
24         value
25     }
26 }
27 // 用来创建 Identifier
28 function createIdentifier(name) {
29     return {
30         type: 'Identifier',
31         name
32     }
33 }
34 // 用来创建 ArrayExpression 节点
35 function createArrayExpression(elements) {
36     return {
37         type: 'ArrayExpression',
38         elements
39     }
40 }
41 // 用来创建 CallExpression 节点
42 function createCallExpression(callee, arguments) {
43     return {
44         type: 'CallExpression',
45         callee: createIdentifier(callee),
46         arguments
47     }
48 }
49
50 // 转换文本节点
51 function transformText(node) {
52     // 如果不是文本节点，则什么都不做
53     if (node.type !== 'Text') return
54     // 文本节点对应的 JavaScript AST 节点其实就是一个字符串字面量，
55     // 因此只需要使用 node.content 创建一个 StringLiteral 类型的节点即可
56     // 最后将文本节点对应的 JavaScript AST 节点添加到 node.jsNode 属性下
57     node.jsNode = createStringLiteral(node.content)
58 }
59 // 转换标签节点
60 function transformElement(node) {
61     // 将转换代码编写在退出阶段的回调函数中
```

```

62 // 这样可以保证该标签节点的子节点全部被处理完毕
63 return () => {
64     // 如果被转换的节点不是元素节点，则什么都不做
65     if (node.type !== 'Element') {
66         return
67     }
68
69     // 1. 创建 h 函数调用语句，
70     // h 函数调用的第一个参数是标签名称，因此我们以 node.tag 来创建一个字符串字面量节
    点作为第一个参数
71     const callExp = createCallExpression('h', [
72         createStringLiteral(node.tag)
73     ])
74     // 2. 处理 h 函数调用的参数
75     node.children.length === 1
76     // 如果当前标签节点只有一个子节点，则直接使用子节点的 jsNode 作为参数
77     ? callExp.arguments.push(node.children[0].jsNode)
78     // 如果当前标签节点有多个子节点，则创建一个 ArrayExpression 节点作为参数
79     : callExp.arguments.push(
80         // 数组的每个元素都是子节点的 jsNode
81         createArrayExpression(node.children.map(c => c.jsNode))
82     )
83     // 3. 将当前标签节点对应的 JavaScript AST 添加到 jsNode 属性下
84     node.jsNode = callExp
85 }
86 }
87 // 转换 Root 根节点
88 function transformRoot(node) {
89     // 将逻辑编写在退出阶段的回调函数中，保证子节点全部被处理完毕
90     return () => {
91         // 如果不是根节点，则什么都不做
92         if (node.type !== 'Root') return
93         // node 是根节点，根节点的第一个子节点就是模板的根节点
94         // 当然，这里我们暂时不考虑模板存在多个根节点的情况
95         const vnodeJSAST = node.children[0].jsNode
96         // 创建 render 函数的声明语句节点，将 vnodeJSAST 作为 render 函数体的返回语句
97         node.jsNode = {
98             type: 'FunctionDecl',
99             id: { type: 'Identifier', name: 'render' },
100             params: [],
101             body: [
102                 {
103                     type: 'ReturnStatement',
104                     return: vnodeJSAST
105                 }
106             ]
107         }

```



```
108     }
109 }
110
111 function traverseNode(ast, context) {
112     // 当前节点, ast 本身就是 Root 节点
113     context.currentNode = ast
114     // 增加退出阶段的回调函数数组
115     const exitFns = []
116
117     // context.nodeTransforms 是一个数组, 其中每一个元素都是一个函数
118     const transforms = context.nodeTransforms || []
119     for (let i = 0; i < transforms.length; i++) {
120         // 转换函数可以返回另外一个函数, 该函数作为退出阶段的回调函数
121         const onExit = transforms[i](context.currentNode, context)
122
123         if (onExit) {
124             // 将退出阶段的回调函数添加到 exitFns 数组中
125             exitFns.push(onExit)
126         }
127
128         // 由于任何转换函数都可能移除当前节点, 因此每个转换函数执行完毕后,
129         // 都应该检查当前节点是否已经被移除, 如果被移除, 直接返回即可
130         if (!context.currentNode) return
131     }
132
133     // 如果有子节点, 则递归地调用 traverseNode 函数进行遍历
134     const children = context.currentNode.children
135     if (children) {
136         children.forEach((cur, i) => {
137             // 设置父节点
138             context.parent = context.currentNode
139             // 设置位置索引
140             context.childIndex = i
141             // 递归调用
142             traverseNode(cur, context)
143         })
144     }
145
146     // 节点处理的最后阶段执行缓存到 exitFns 中的回调函数
147     // tip: 这里我们要反序执行
148     let i = exitFns.length
149     while (i--) {
150         exitFns[i]()
151     }
152 }
153
154 function transform(ast) {
```

```

155 // 在 transform 函数内创建 context 对象
156 const context = {
157   // 增加 currentNode, 存储当前正在转换的节点
158   currentNode: null,
159   // 增加 childIndex, 存储当前节点在父节点的 children 中的位置索引
160   childIndex: 0,
161   // 增加 parent, 存储当前转换节点的父节点
162   parent: null,
163   // 用于替换节点的函数, 接收新节点作为参数
164   replaceNode(node) {
165     // 为了替换节点, 我们需要修改 AST
166     // 找到当前节点在父节点的 children 中的位置: context.childIndex
167     // 然后使用新节点替换即可
168     context.parent.children[context.childIndex] = node
169     // 由于当前新节点已经被新节点替换掉, 因此我们需要将 currentNode 更新为新节点
170     context.currentNode = node
171   },
172   // 删除当前节点
173   removeNode() {
174     if (context.parent) {
175       // 调用数组的 splice 方法, 根据当前节点的索引删除当前节点
176       context.parent.children.splice(context.childIndex, 1)
177       // 将 context.currentNode 置空
178       context.currentNode = null
179     }
180   },
181   // 注册 nodeTransforms 数组
182   nodeTransforms: [
183     transformElement,
184     transformText,
185     transformRoot,
186   ]
187 },
188
189 // 调用 traverseNode 完成转换
190 traverseNode(ast, context)
191 // 打印 AST 信息
192 dump(ast)
193 }
194
195 module.exports = {
196   transform
197 }

```

```

1  function genNodeList(nodes, context) {
2      const { push } = context
3      for (let i = 0; i < nodes.length; i++) {
4          const node = nodes[i]
5          genNode(node, context)
6          if (i < nodes.length - 1) {
7              push(', ')
8          }
9      }
10 }
11
12 function genFunctionDecl(node, context) {
13     // 从 context 对象中取出工具函数
14     const { push, indent, deIndent } = context
15     // node.id 是一个标识符，用来描述函数的名称，即 node.id.name
16     push(`function ${node.id.name} `)
17     push(`(`)
18     // 调用 genNodeList 为函数的参数生成代码
19     genNodeList(node.params, context)
20     push(`)` )
21     push(`{` )
22     // 缩进
23     indent()
24     // 为函数体生成代码，递归地调用 genNode 函数
25     node.body.forEach(n => genNode(n, context))
26     // 取消缩进
27     deIndent()
28     push(`}` )
29 }
30
31 function genArrayExpression(node, context) {
32     const { push } = context
33     // 追加方括号
34     push(`[`)
35     // 调用 genNodeList 为数组元素生成代码
36     genNodeList(node.elements, context)
37     // 补全方括号
38     push(`]`)
39 }
40
41 function genReturnStatement(node, context) {
42     const { push } = context
43     // 追加 return 关键字和空格
44     push(`return `)
45     // 调用 genNode 函数递归生成返回值代码
46     genNode(node.return, context)
47 }

```

```

48
49 function genStringLiteral(node, context) {
50     const { push } = context
51     // 对于字符串字面量，只需要追加与 node.value 对应的字符串即可
52     push(`${node.value}`)
53 }
54
55 function genCallExpression(node, context) {
56     const { push } = context
57     // 取得被调用函数名称和参数列表
58     const { callee, arguments: args } = node
59     // 生成函数调用代码
60     push(`${callee.name}(`)
61     // 调用 genNodeList 生成参数代码
62     genNodeList(args, context)
63     // 补全括号
64     push(``)
65 }
66
67 function genNode(node, context) {
68     switch (node.type) {
69         case 'FunctionDecl':
70             genFunctionDecl(node, context)
71             break;
72         case 'ReturnStatement':
73             genReturnStateMent(node, context)
74             break;
75         case 'CallExpression':
76             genCallExpression(node, context)
77             break;
78         case 'StringLiteral':
79             genStringLiteral(node, context)
80             break;
81         case 'ArrayExpression':
82             genArrayExpression(node, context)
83             break;
84     }
85 }
86
87 function genertae(node) {
88     const context = {
89         // 存储最终生成的渲染函数代码
90         code: '',
91         // 生成代码时，通过调用 push 函数完成代码拼接
92         push(code) {
93             context.code += code
94         },

```

```

95     // 当前缩进级别，初始值为 0，即没有缩进
96     currentIndent: 0,
97     // 该函数用来换行，即在代码字符串的后面追加 \n 字符
98     // 另外，换行时应该保留缩进，所以我们还要追加 currentIndent * 2 个空格字符
99     newLine() {
100         context.code += '\n' + ' '.repeat(context.currentIndent)
101     },
102     // 用来缩进，即让 currentIndent 自增后，调用换行函数
103     indent() {
104         context.currentIndent++
105         context.newLine()
106     },
107     // 取消缩进，即让 currentIndent 自减后，调用换行函数
108     deIndent() {
109         context.currentIndent--
110         context.newLine()
111     }
112 }
113
114 // 调用 genNode 函数完成代码生成工作
115 genNode(node, context)
116
117 // 返回渲染函数代码
118 return context.code
119 }
120
121 module.exports = {
122     genertae
123 }

```

使用编译器

```

1  const { parse } = require('./compiler/parse')
2  const { transform } = require('./compiler/transform')
3  const { generate } = require('./compiler/generate')
4
5  function compiler(template) {
6      // 模板 AST
7      const ast = parse(template)
8      // 将模板 AST 转换为 javascript AST
9      transform(ast)
10     // 代码生成
11     const code = generate(ast.jsNode)

```

```
12     return code
13   }
14
15   const targetAST = compiler('<div><p>Vue</p><p>Template</p></div>')
16
17   console.log(targetAST)
```

```
1  <div>
2    <p>Vue</p>
3    <p>Template</p>
4  </div>
```

假设我们有如上模板，很显然我们函数是无法使用它的，我们必须把它转换为函数 `h` 形式，如下：

```
1  function render() {
2    return h('div', [
3      h('p', 'Vue'),
4      h('p', 'Template')
5    ])
6  }
```